第10章 排序

提纲

CONTENTS

10.1 排序的基本概念

10.2 插入排序

10.3 交换排序

10.4 选择排序

10.5 归并排序

10.6 基数排序

10.7 各种内排序方法的比较和选择

10.8 外排序

10.1 排序的基本概念

1．什么是排序

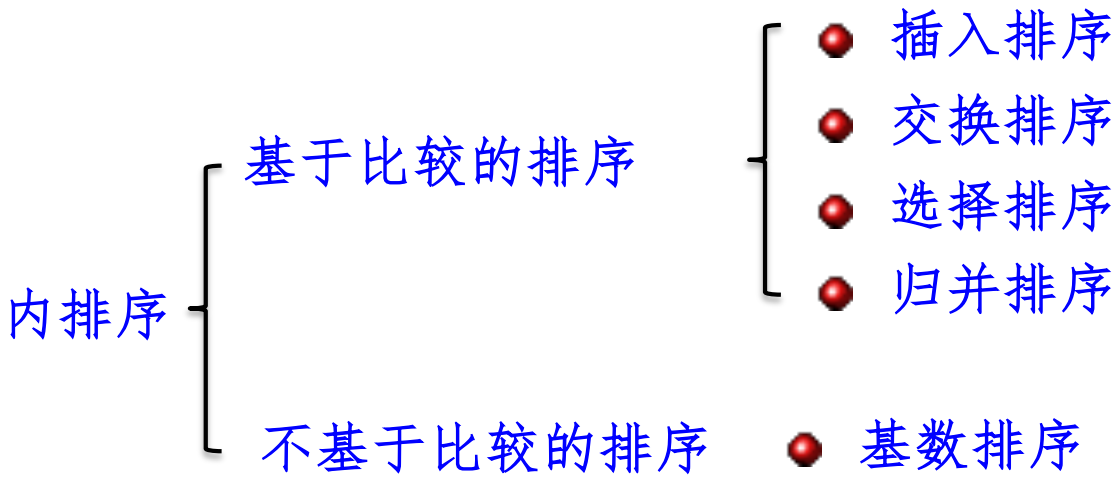
- 排序：整理表中的元素，使之按关键字递增或递减有序排列。
- 本章仅讨论递增排序的情况，在默认情况下均指递增排序。
- 排序的输入输出如下：

- **输入**：n个元素序列为R_0、R_1、…、R_{n-1}，其相应的关键字分别为k_0、k_1、…、k_{n-1}。
- **输出**：R_{i0}，R_{i1}，…，R_{in-1} ，使得$k_{i0} \leqslant k_{i1} \leqslant \cdots \leqslant k_{in-1}$。

2．内排序和外排序

- 在排序过程中，若整个表都是放在内存中处理，排序时不涉及数据的内、外存交换，则称之为内排序。
- 反之，若排序过程中要进行数据的内、外存交换，则称之为外排序。

3．内排序的分类



4．基于比较的排序算法的性能

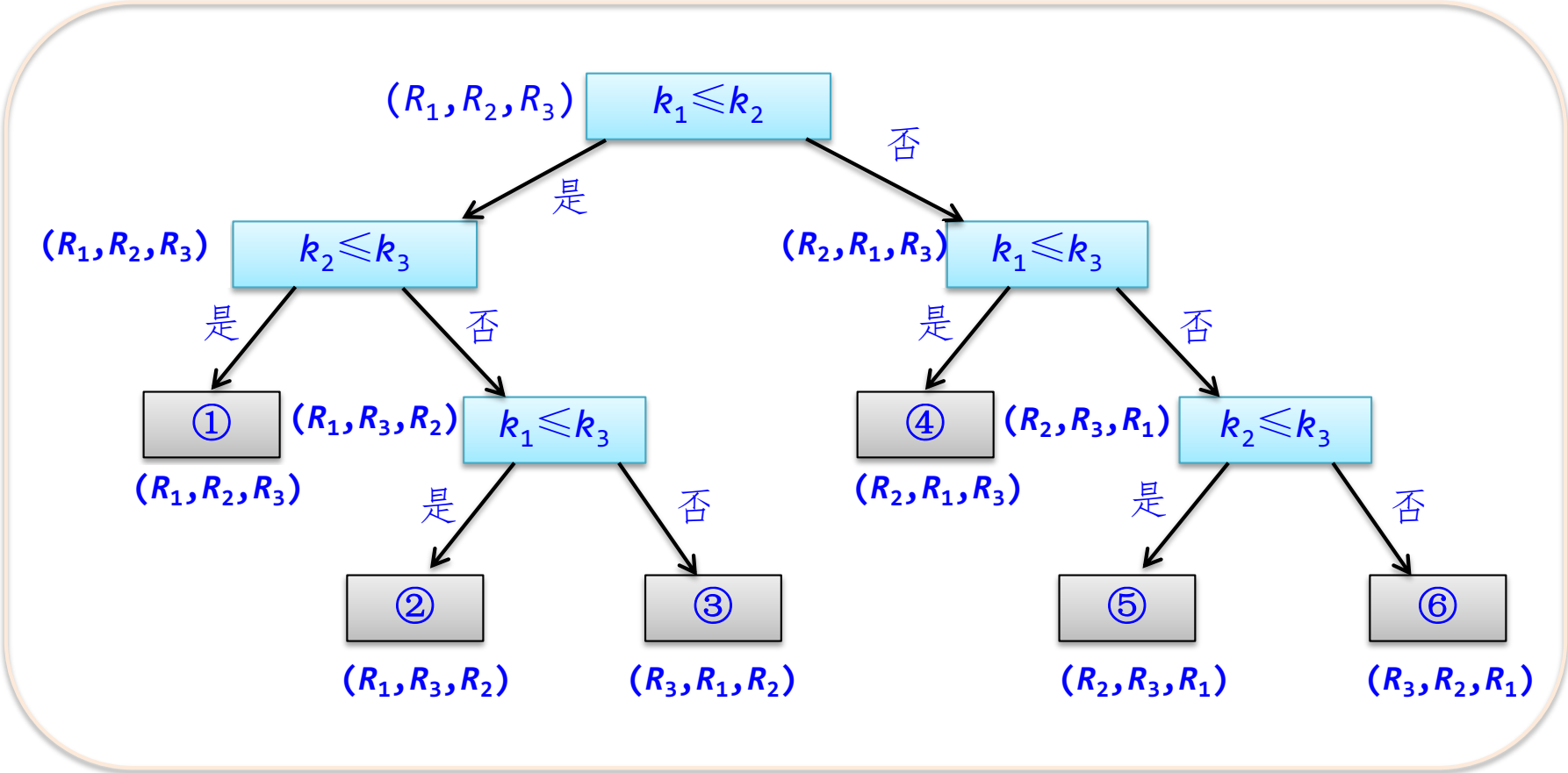
基于比较的排序算法中，主要进行以下两种基本操作：

- 比较：元素关键字之间的比较。
- 移动：元素从一个位置移动到另一个位置。

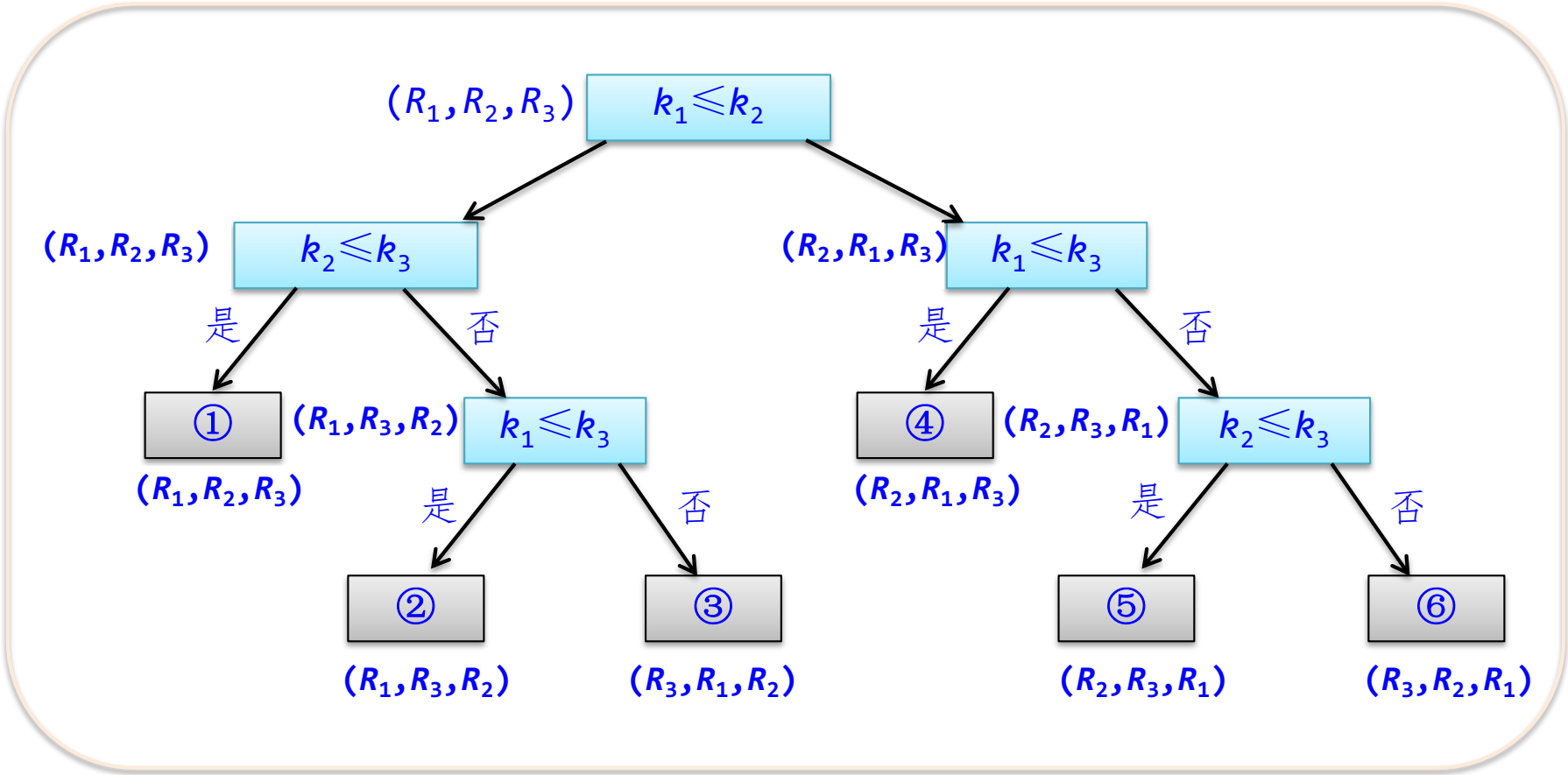
- 排序算法的性能由算法的时间和空间确定的，而时间又是由比较和移动的次数确定的。
- 若待排序元素的关键字顺序正好和排序顺序相同，称此表中元素为正序。
- 反之，若待排序元素的关键字顺序正好和排序顺序相反，称此表中元素为反序。

基于比较的排序算法的下界：$T(n)\geqslant O(n\log_2 n)$

n=3的决策树



正序不变，逆序交换！ 3!=6个叶子结点



推广为n个元素（假设关键字均不同）

- **排序算法所用的比较次数等于决策树中最深的叶子结点的深度。**
- **所用的平均比较次数等于决策树中叶子结点的平均深度即树的高度。**

- n个元素排序结果有$n!$种情况，对应的决策树是一棵有$n!$个叶子结点的二叉树。
- 设其高度为h，其中没有单分支结点，总结点个数$=2n!-1$，其高度等同于含$2n!-1$个结点的完全二叉树的高度，则$h=\lceil \log_2(2n!) \rceil = \log_2 n!+1$，而$\log_2 n! \approx n\log_2 n$，即$h=O(n\log_2 n)$。
- 排序中移动次数与比较次数属于同数量级。
- n个元素采用基于比较的排序方法平均时间复杂度为$O(n\log_2 n)$，最坏情况下的时间下界也为$O(n\log_2 n)$。

基于比较排序算法的平均时间复杂度不可能优于$O(n\log_2 n)$

5．排序的稳定性

- 当待排序元素的关键字均不相同时，排序的结果是唯一的，否则排序的结果不一定唯一。
- 如果待排序的表中，存在有多个关键字相同的元素，经过排序后这些具有相同关键字的元素之间的相对次序保持不变，则称这种排序方法是稳定的。
- 反之，若具有相同关键字的元素之间的相对次序发生变化，则称这种排序方法是不稳定的。

6．排序数据的组织

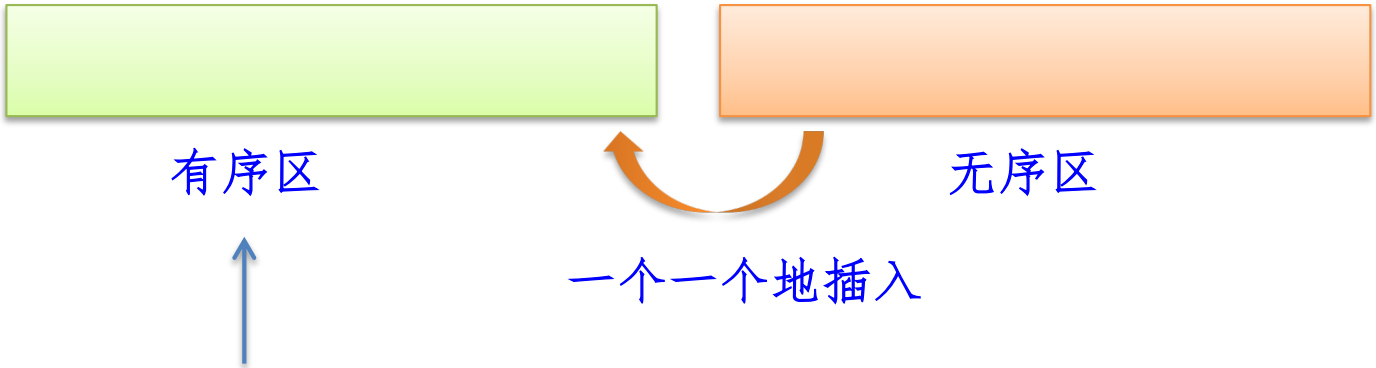
- 以顺序表作为排序表的存储结构（除基数排序采用单链表外）。假设关键字为int类型，待排序的顺序表直接采用vector<int>向量*R*表示。
- 如10个整数关键字序列表示为：*R*={1,6,2,5,3,7,4,8,2,4}（存在相同的关键字）。
- 若待排序表中每个元素除整数关键字外还有其他数据项，可以采用相应类型的vector<T>向量表示。
- 如3个学生元素，每个元素由学号和姓名组成，*R*表示为：*R*={[1,"Mary"],[3,"John"],[2,"Smith"]}。

各种内排序方法的比较和选择

排序方法	时间复杂度			空间复杂度	稳定性
	平均情况	最坏情况	最好情况		
直接插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
折半插入排序	$O(n^2)$	$O(n^2)$	$O(n\log_2 n)$	$O(1)$	稳定
希尔排序	$O(n^{1.58})$			$O(1)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(\log_2 n)$	不稳定
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定
二路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	稳定
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	稳定

10.2 插入排序

基本思路



不是全局有序（在后面排序中可能发生位置的改变）

主要的插入排序方法：

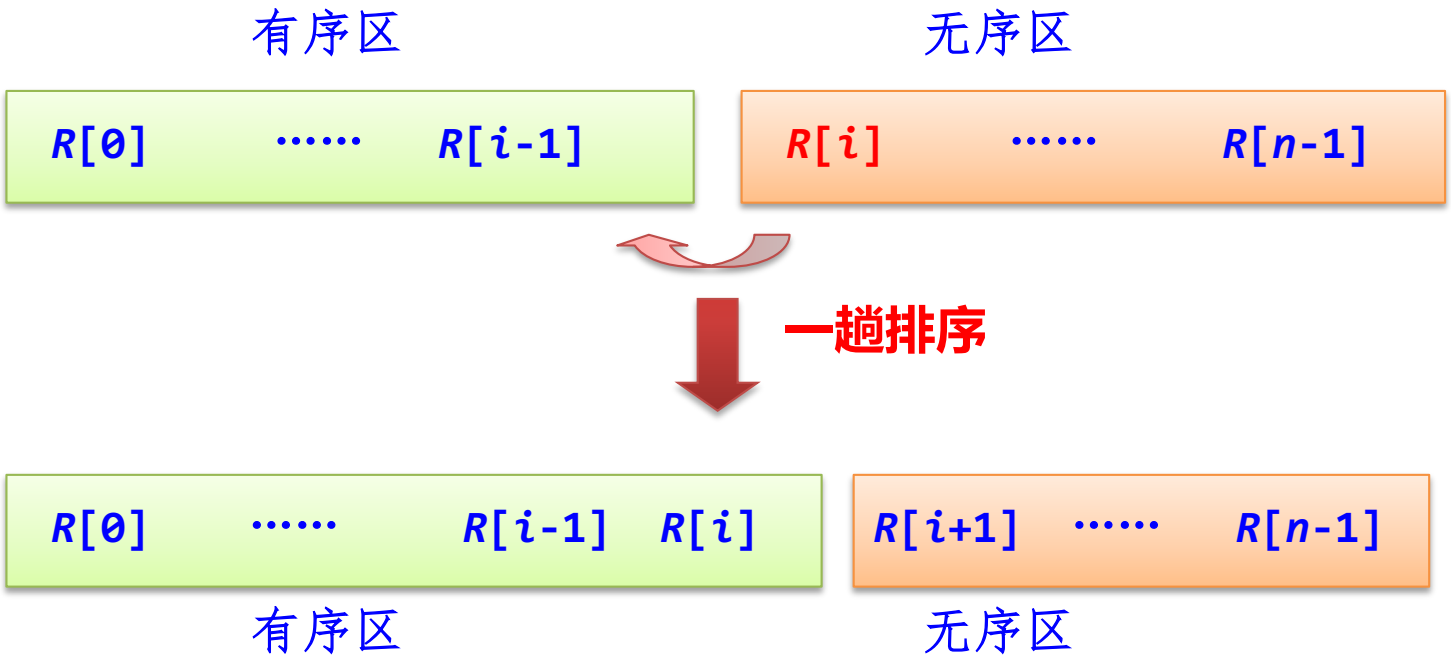
（1）直接插入排序

（2）折半插入排序

（3）希尔排序

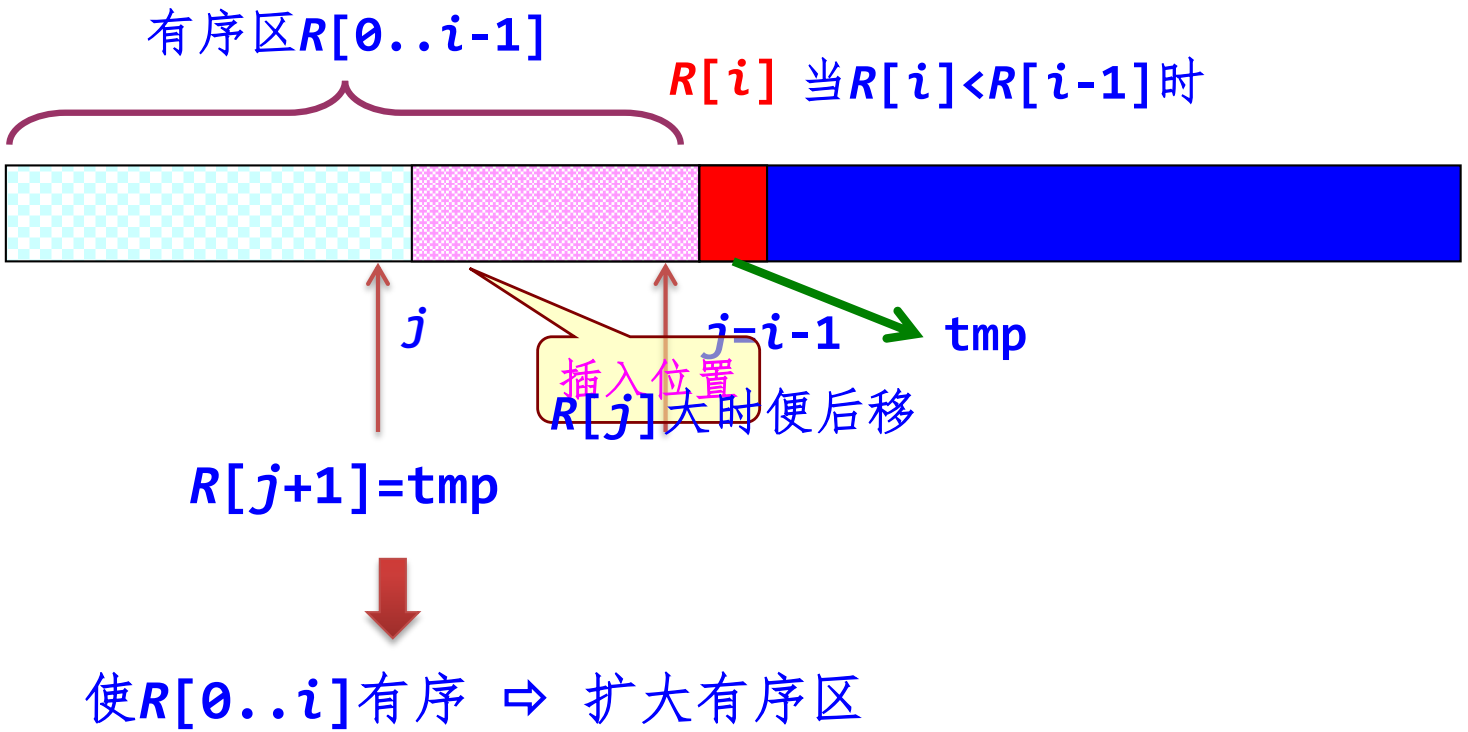
10.2.1 直接插入排序

1. 排序思路



- 初始时，有序区只有一个元素R[0]
- i＝1~n-1，共经过n-1趟排序

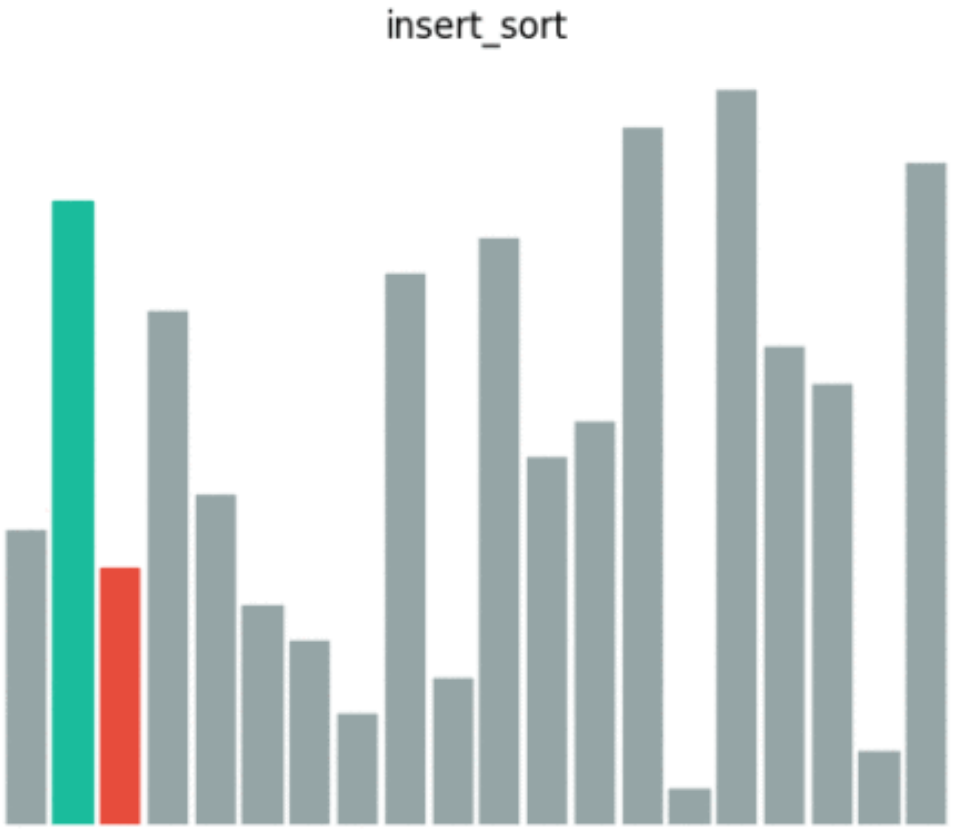
一趟直接插入排序：在有序区中插入R[i]的过程。



【例10.1】 设待排序表有10个元素，其关键字序列为（9，8，7，6，5，4，3，2，1，0）。说明采用直接插入排序方法进行排序的过程。

初始	[9]	8	7	6	5	4	3	2	1	0
i=1:	[8	9]	7	6	5	4	3	2	1	0
i=2:	[7	8	9]	6	5	4	3	2	1	0
i=3:	[6	7	8	9]	5	4	3	2	1	0
i=4:	[5	6	7	8	9]	4	3	2	1	0
i=5:	[4	5	6	7	8	9]	3	2	1	0
i=6:	[3	4	5	6	7	8	9]	2	1	0
i=7:	[2	3	4	5	6	7	8	9]	1	0
i=8:	[1	2	3	4	5	6	7	8	9]	0
i=9:	[0	1	2	3	4	5	6	7	8	9]

直接插入排序动画



2．排序算法

```
void InsertSort(vector<int>& R,int n)          //直接插入排序
{  for (int i=1;i<n;i++)                       //从R[1]开始
   {  if (R[i]<R[i-1])                         //反序时
      {  int tmp=R[i];                         //取出无序区的第一个元素
         int j=i-1;            //有序区R[0..i-1]中向前找R[i]的插入位置
         do
         {  R[j+1]=R[j];                       //将大于tmp的元素后移
            j--;                               //继续向前比较
         } while (j>=0 && R[j]>tmp);
         R[j+1]=tmp;                           //在j+1处插入R[i]
      }
   }
}
```

向前找到第一个≤tmp的R[j]，在其后插入tmp

【算法扩展】在排序中通常利用“<”比较实现递增排序，可以自定义比较函数cmp(x,y)，如x<y时，该函数返回true，否则返回false，这样将上述算法中R[j]<=tmp转换为!tmp<R[j]即!cmp(tmp，R[j])，

```
bool cmp(int x,int y)     //实现递增排序的自定义比较函数
{   if (x<y) return true;
    else return false;
}
```

递增排序R

```
void InsertSort(vector<int>& R,int n)          //直接插入排序（递增）
{  for (int i=1;i<n;i++)                       //从R[1]开始
   {  if (cmp(R[i],R[i-1]))                    //反序时
      {  int tmp=R[i];                         //取出无序区的第一个元素
         int j=i-1;            //有序区R[0..i-1]中向前找插入位置
         do
         {  R[j+1]=R[j];                      //将大于tmp的元素后移
            j--;                              //继续向前比较
         } while (j>=0 && !cmp(R[j],tmp));
         R[j+1]=tmp;                          //在j+1处插入R[i]
      }
   }
}
```

```
bool cmp(int x,int y)   //实现递增排序的自定义比较函数
{  if (x<y) return true;
   else return false;
}
```

```
bool cmp(int x,int y)   //实现递减排序的自定义比较函数
{  if (x>y) return true;
   else return false;
}
```

- InsertSort算法不变即可递减排序R
- 后面讨论的各种排序算法都可以这样转换为递减排序或者按定制的方式排序

3．算法分析

1）最好情况分析

初始数据序列正序

$$C_{\min}=\sum_{i=1}^{n-1}1=n-1=O(n),\quad M_{\min}=0$$

最好情况下的时间复杂度为**O(*n*)**。

2）最坏情况分析

初始数据序列反序

$$C_{\max}=\sum_{i=1}^{n-1} i=\frac{n(n-1)}{2}=O(n^2)$$

$$M_{\max}=\sum_{i=1}^{n-1}(i+2)=\frac{(n-1)(n+4)}{2}=O(n^2)$$

最坏情况下的时间复杂度为$O(n^2)$。

3）平均情况分析

将**R[i]**插入到**R[0..i-1]**（含**i**个元素）的中间位置

$$C_{\text{avg}} = \sum_{i=1}^{n-1} \left(\frac{i}{2}\right) = \frac{n(n-1)}{4} = O(n^2)$$

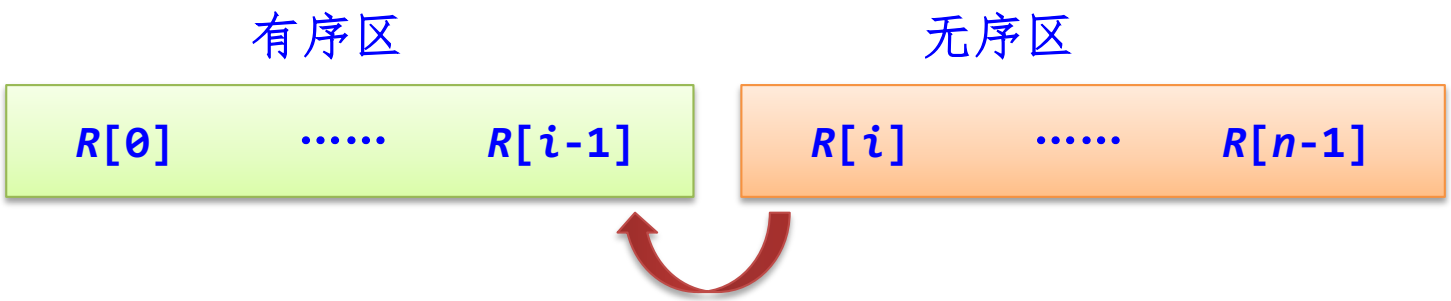
$$M_{\text{avg}} = \sum_{i=1}^{n-1} \left(\frac{i}{2} + 2\right) = O(n^2)$$

平均情况的时间复杂度为$O(n^2)$。

10.2.2 折半插入排序

1. 排序思路

查找采用折半查找方法，称为二分插入排序或折半插入排序。



采用折半查找在有序区找到插入的位置

2．排序算法

```
void BinInsertSort(vector<int>& R,int n)      //折半插入排序
{  for (int i=1;i<n;i++)                      //从R[1]开始
   {  if (R[i]<R[i-1])                        //反序时
      {  int tmp=R[i];                        //将R[i]保存到tmp中
         int low=0,high=i-1;
         while (low<=high)                    //折半查找tmp的插入点high+1
         {  int mid=(low+high)/2;             //取中间位置
            if (tmp<R[mid])
               high=mid-1;                    //插入点在左半区
            else
               low=mid+1;                     //插入点在右半区
         }
         for (int j=i-1;j>=high+1;j--)        //元素后移
            R[j+1]=R[j];
         R[high+1]=tmp;                       //原R[i]插入到R[high+1]中
      }
   }
}
```

折半查找与lower_bound算法类似

3．算法分析

在任何情况下排序中元素移动的次数与直接插入排序的相同，不同的仅是变分散移动为集中移动。

$$\sum_{i=1}^{n-1}(\log_2(i+1)-1+\frac{i}{2}+2)=O(n^2)$$

平均情况的时间复杂度为$O(n^2)$。

【例（补充）】对同一待排序序列分别进行折半插入排序和直接插入排序，两者之间可能的不同之处是（　）。

A.排序的总趟数

B.元素的移动次数

C.使用辅助空间的数量

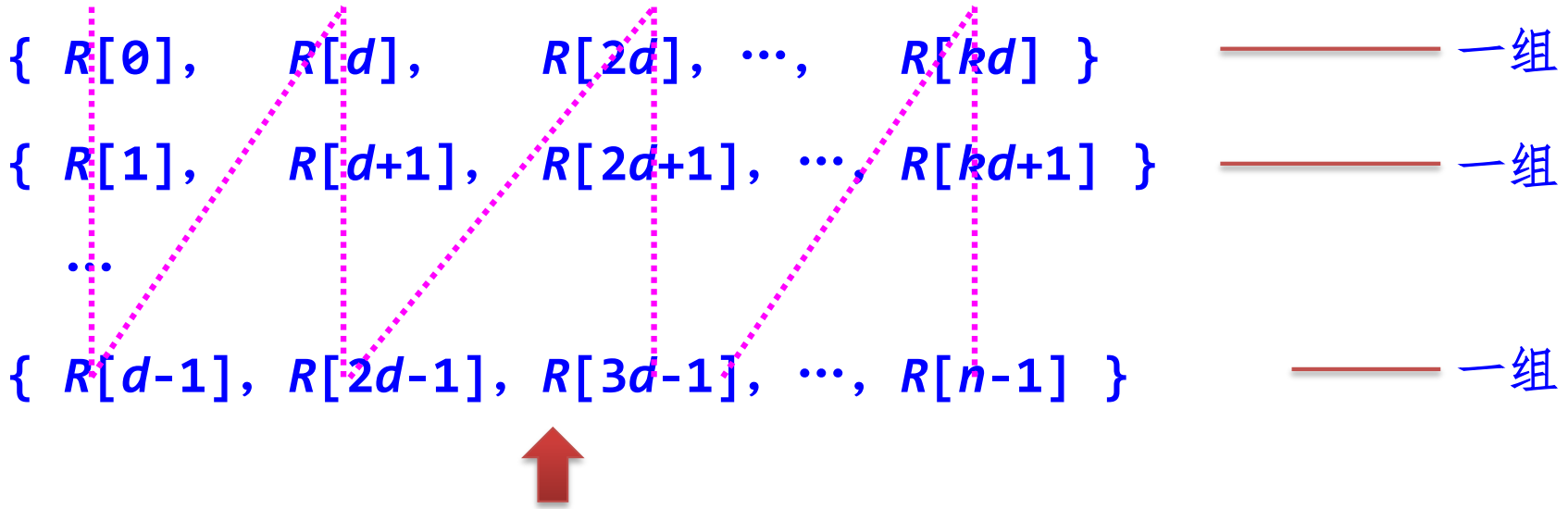
D.元素之间的比较次数

说明：本题为2012年全国考研题

10.2.3 希尔排序

1. 排序思路

例如：将*n*个元素分成*d*个组：



① $d=n/2$

② 将排序序列分为d个组，在各组内进行直接插入排序

③ 递减$d=d/2$，重复② ，直到$d=0$为止

算法最后一趟（d=1）对所有元素进行了直接插入排序，所以结果一定是正确的。

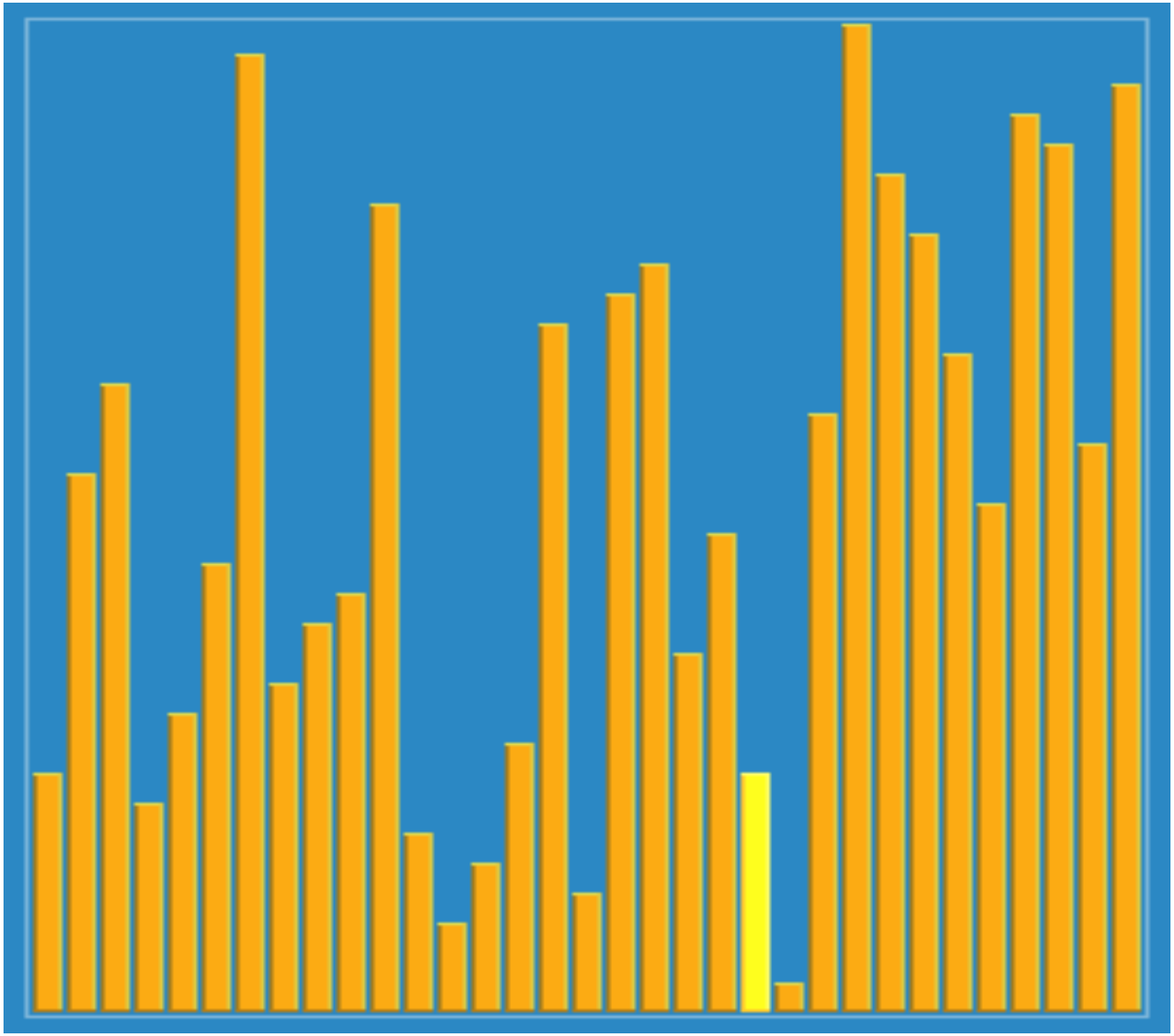
d序列：d=n/2开始，每次d=d/2，直到d=0

【例10.2】 设待排序的表有10个元素，其关键字分别为（9，8，7，6，5，4，3，2，1，0）。说明采用希尔排序方法进行排序的过程。

初始序列	9	8	7	6	5	4	3	2	1	0
d=5	9	8	7	6	5	4	3	2	1	0
直接插入排序	4	3	2	1	0	9	8	7	6	5
d=*d*/2=2	4	3	2	1	0	9	8	7	6	5
直接插入排序	0	1	2	3	4	5	6	7	8	9
d=*d*/2=1	0	1	2	3	4	5	6	7	8	9
直接插入排序	0	1	2	3	4	5	6	7	8	9

对于*d*=1的一趟，排序前的数据已将近正序！

希尔排序动画



2．排序算法

取$d_1=n/2$，$d_{i+1}=\lfloor d_i/2 \rfloor$（Shell增量序列）时的希尔排序的算法

```
void ShellSort(vector<int>& R,int n)          //希尔排序
{  int d=n/2;                                 //增量置初值
   while (d>0)
   {  for (int i=d;i<n;i++)                   //相隔d位置的元素组直接插入排序
      {  if (R[i]<R[i-d])                     //反序时
         {  int tmp=R[i];
            int j=i-d;
            do
            {  R[j+d]=R[j];                   //将大于tmp的元素同组中后移
               j=j-d;                         //继续向前比较
            } while (j>=0 && R[j]>tmp);
            R[j+d]=tmp;                       //在j+d处插入R[i]
         }
      }
      d=d/2;                                  //减小增量
   }
}
```

3．算法分析

- $d_1=n/2$，$d_{i+1}=\lfloor d_i/2 \rfloor$（$i \geqslant 1$），也就是说，每趟后一个增量是前一个增量的1/2，则经过$t=\lceil \log_2 n \rceil-1$趟后$d_t=1$，再经过一趟最后直接插入排序使整数数序变为有序的。
- 希尔算法的时间复杂度难以分析，一般认为其平均时间复杂度为$O(n^{1.58})$。希尔排序的速度通常要比直接插入排序快。

例如：有10个元素要排序。

直接插入排序

大约时间=10^2=100

希尔排序

d=5：分为5组，时间约为5×2^2 ＝ 20

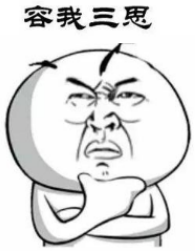
＋

d=2：分为2组，时间约为2×5^2 ＝ 50

＋

d=1：分为1组，几乎有序，时间约为10

＝ 80



希尔排序是一种不稳定的排序算法

n=10,*d*=2	3	5	10	8	7	2	8	1	20	6
第1组排序结果	3		7		8		10		20	
第2组排序结果		1		2		5		6		8
d=2趟排序结果	3	1	7	2	8	5	10	6	20	8

相对位置发生改变

一般地，相距位置较大的两个元素发生交换 ⇨ 不稳定！

20. 【2015 统考真题】希尔排序的组内排序采用的是（ ）。

A. 直接插入排序　B. 折半插入排序　C. 快速排序　D. 归并排序

21. 【2018 统考真题】对初始数据序列(8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6)进行希尔排序。若第一趟排序结果为(1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8)，第二趟排序结果为(1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9)，则两趟排序采用的增量（间隔）依次是（　）。

A. 3, 1　　B. 3, 2　　C. 5, 2　　D. 5, 3

19.【2014 统考真题】用希尔排序算法对一个数据序列进行排序时，若第一趟排序结果为 9, 1, 4, 13, 7, 8, 20, 23, 15，则该趟排序采用的增量（间隔）可能是（　）。

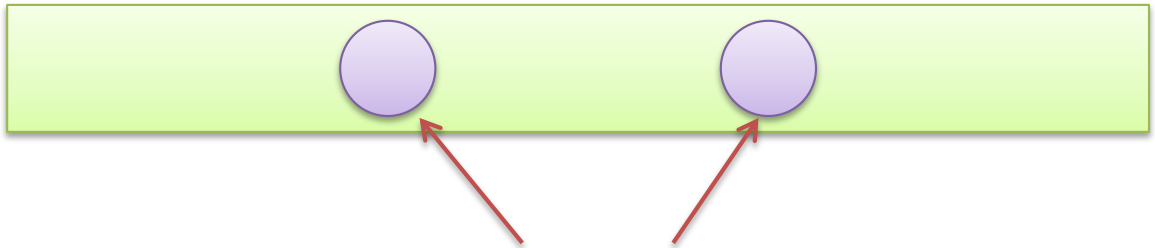
A. 2　　B. 3　　C. 4　　D. 5

各种内排序方法的比较和选择

排序方法	时间复杂度			空间复杂度	稳定性
	平均情况	最坏情况	最好情况		
直接插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
折半插入排序	$O(n^2)$	$O(n^2)$	$O(n\log_2 n)$	$O(1)$	稳定
希尔排序	$O(n^{1.58})$			$O(1)$	不稳定

10.3 交换排序

基本思路



两个元素反序时进行交换

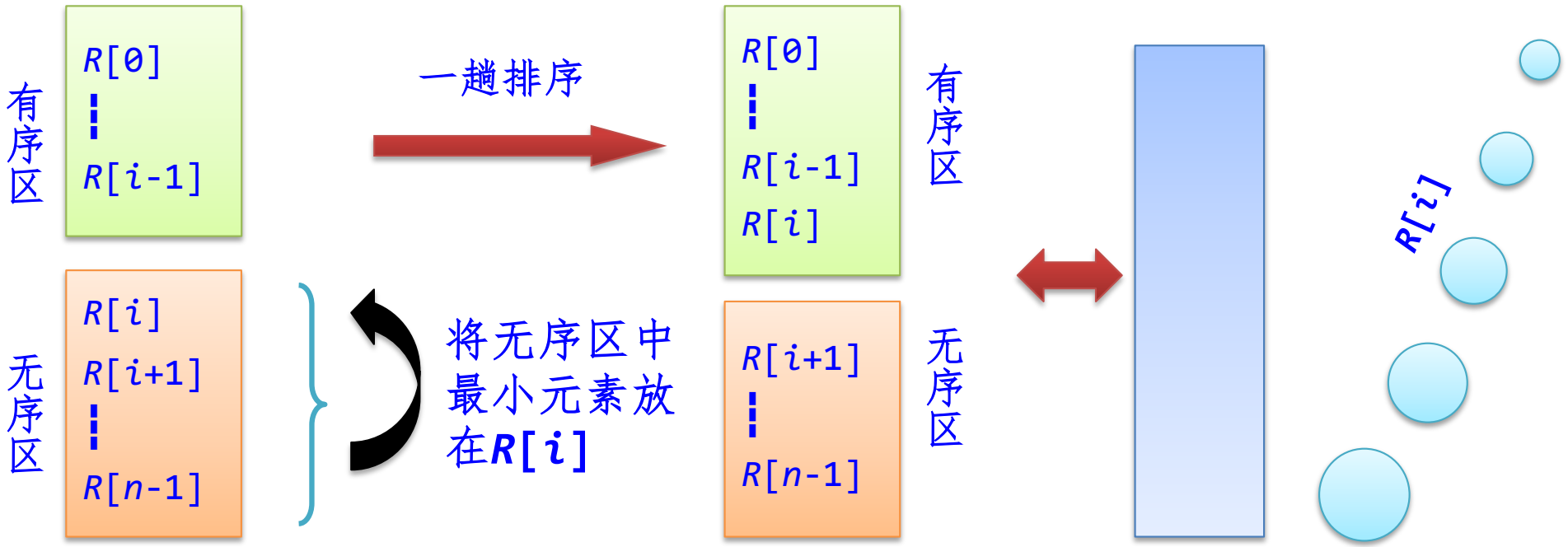
主要的交换排序方法：

（1）冒泡排序

（2）快速排序

10.3.1 冒泡排序

1．排序思路



初始有序区为空。

$i=0\sim n-2$，共$n-1$趟使整个数据有序。

→ 有序区总是全局有序的

【例10.3】 设待排序的表有10个元素，其关键字分别为{9，8，7，6，5，4，3，2，1，0}。说明采用冒泡排序方法进行排序的过程。

初始：　[ ]9　8　7　6　5　4　3　2　1　0

i=0：　[0] 9　8　7　6　5　4　3　2　1

i=1：　[0　1] 9　8　7　6　5　4　3　2

i=2：　[0　1　2] 9　8　7　6　5　4　3

i=3：　[0　1　2　3] 9　8　7　6　5　4

i=4：　[0　1　2　3　4] 9　8　7　6　5

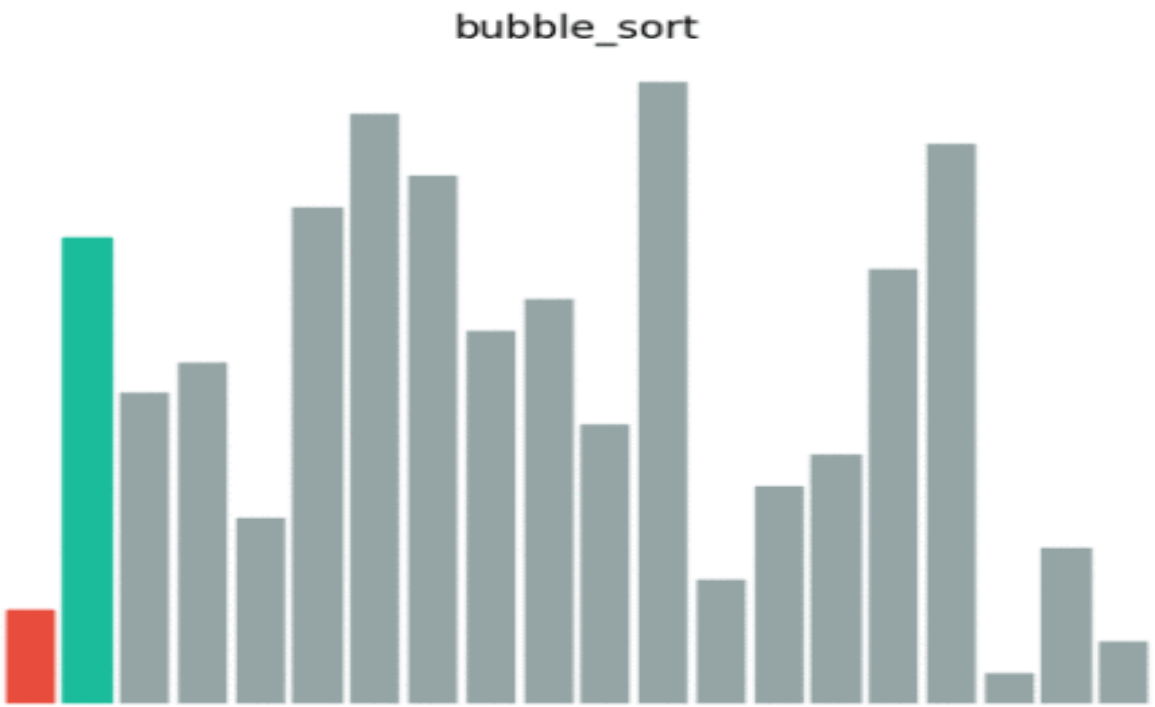
i=5：　[0　1　2　3　4　5] 9　8　7　6

i=6：　[0　1　2　3　4　5　6] 9　8　7

i=7：　[0　1　2　3　4　5　6　7] 9　8

i=8：　[0　1　2　3　4　5　6　7　8] 9

冒泡排序动画



2．排序算法

在冒泡排序算法，若某一趟没有出现任何元素交换，说明所有元素已排好序了，就可以结束本算法。

```
void BubbleSort(vector<int>& R,int n)          //冒泡排序
{  for (int i=0;i<n-1;i++)
   {  bool exchange=false;                     //本趟前将exchange置为false
      for (int j=n-1;j>i;j--)                  //在无序区找出最小元素
          if (R[j]<R[j-1])                     //反序时交换
          {  swap(R[j],R[j-1]);
             exchange=true;                    //本趟发生交换置exchange为true
          }
      if (!exchange)                           //本趟没有发生交换，中途结束算法
         return;
   }
}
```

3．算法分析

1）最好情况分析

初始数据序列正序

$$C_{\min}=\sum_{i=0}^{n-2}1=n-1=O(n),\quad M_{\min}=0$$

最好情况下的时间复杂度为**O(*n*)**。

2）最坏情况分析

初始数据序列反序

$$C_{\max}=\sum_{i=0}^{n-2}(n-i-1)=\frac{n(n-1)}{2}=O(n^2)$$

$$M_{\max}=\sum_{i=0}^{n-2}3(n-i-1)=\frac{3n(n-1)}{2}=O(n^2)$$

最坏情况下的时间复杂度为$O(n^2)$。

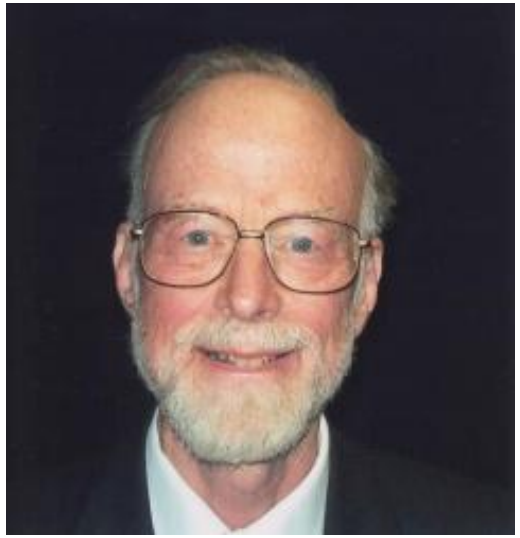
3）平均情况分析

- 算法可能在中间的某一趟排序完成后就结束，但平均的排序趟数仍是$O(n)$。
- 每一趟的关键字比较次数和元素移动次数为$O(n)$，所以平均时间复杂度为$O(n^2)$。

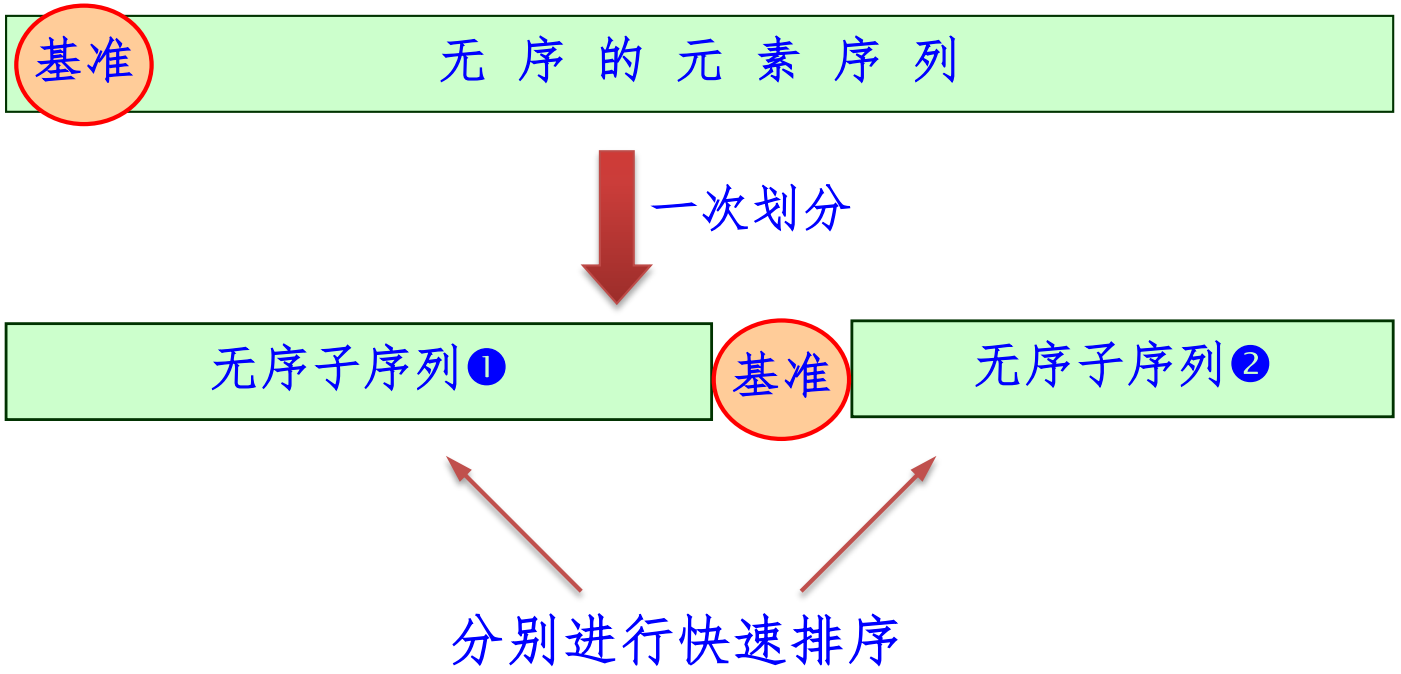
平均情况的时间复杂度为$O(n^2)$。

10.3.2 快速排序

1. 排序思路



- 1960年发布了使他闻名于世的快速排序算法（Quick Sort），这个算法也是当前世界上使用最广泛的算法之一
- 领导了Algol 60第一个商用编译器的设计与开发
- 从1977年开始，Tony Hoare博士任职于牛津大学，投身于计算系统的精确性的研究、设计及开发
- 1980年获得图灵奖
- 2000年Hoare因为其在计算机科学与教育上做出的贡献被封为爵士。



每趟使表的第1个元素放入适当位置（归位），将表一分为二，对子表按递归方式继续这种划分，直至划分的子表长为0或1（递归出口）。

对无序区R[s..t]快速排序的递归模型如下：

f(R, s, t) ≡ 不做任何事情　　当R[s..t]为空或者仅有一个元素时

f(R, s, t) ≡ 划分后基准位置为i;　　其他情况

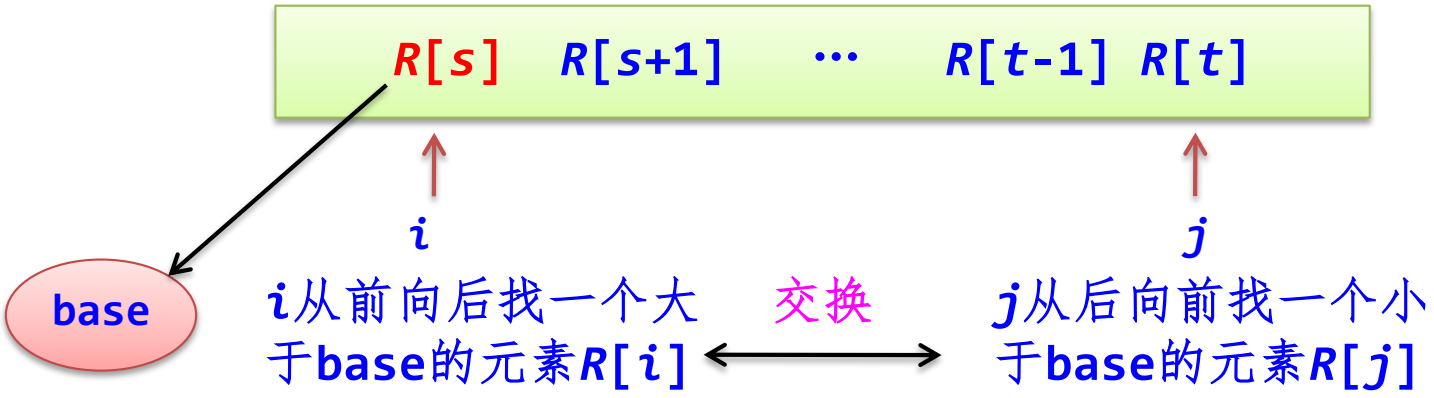
f(R, s, i-1);

f(R, i+1, t);

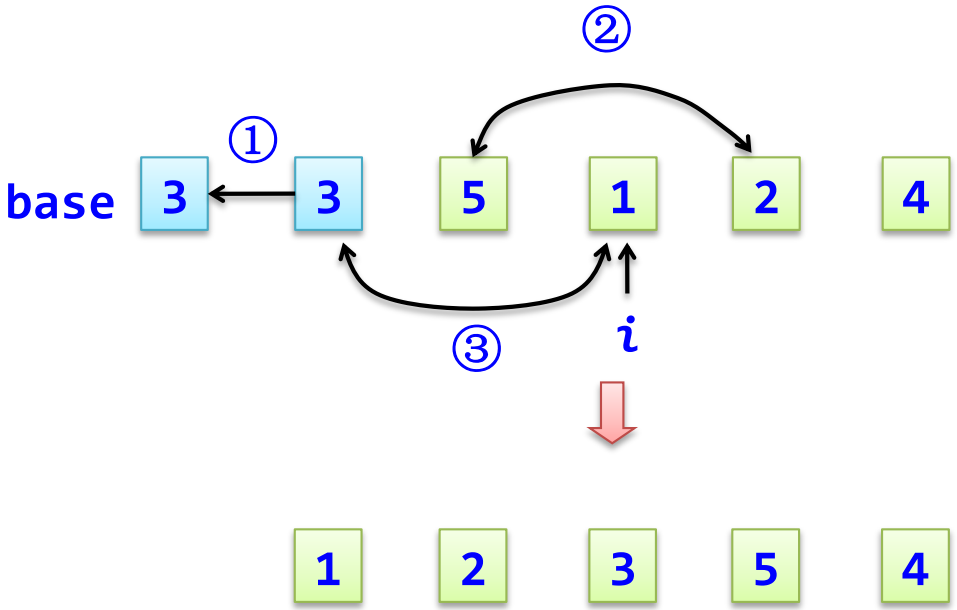
2．排序算法

1）划分算法设计

方法1



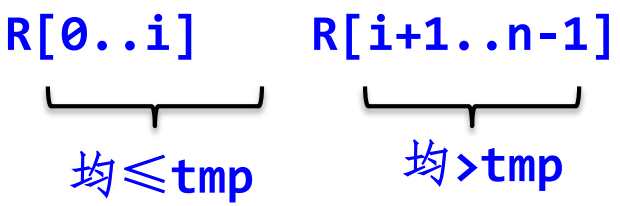
直到i=j，再将基准R[s]和R[i]交换



说明

每次i、j位置的元素都与base比较：重复

```
int Partition1(vector<int>& R,int s,int t)    //划分算法1
{  int base=R[s];                             //以表首元素为基准
   int i=s,j=t;
   while (i<j)                                //从表两端交替向中间遍历,直至i=j为止
   {  while (i<j && R[j]>base)
         j--;                                 //后→前找一个小于等于基准的R[j]
      while (i<j && R[i]<=base)
         i++;                                 //前→后找一个大于基准的R[i]
      if (i<j)
         swap(R[i],R[j]);                     //将R[i]和R[j]进行交换
   }
   swap(R[s],R[i]);                           //将基准R[s]和R[i]进行交换
   return i;
}
```



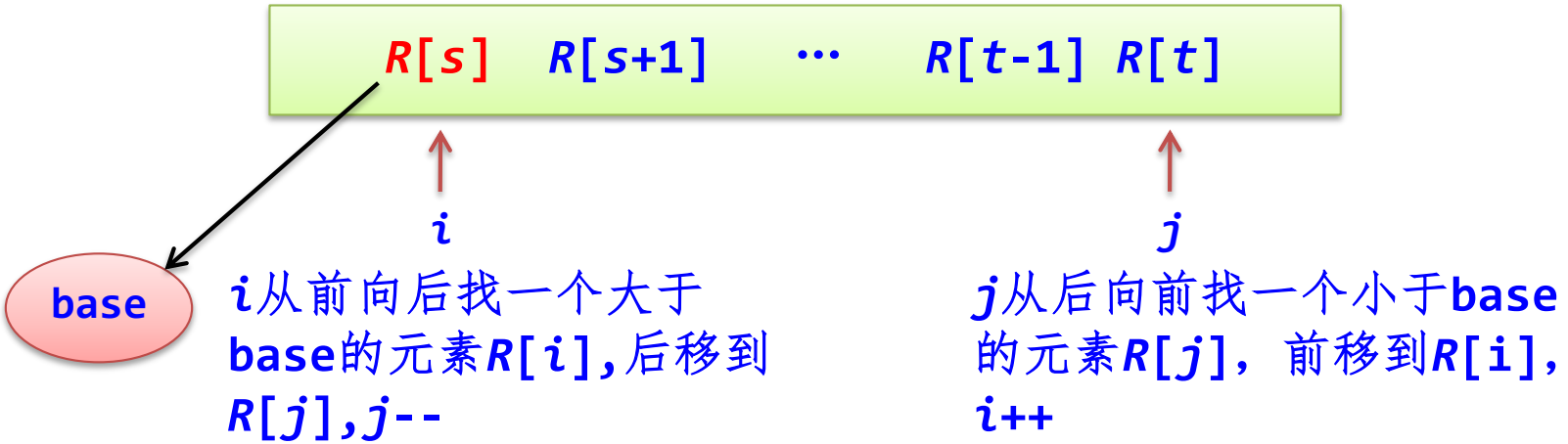
消除重复比较后的优化算法

```
int Partition1_1(vector<int>& R,int s,int t) //划分算法1的改进版本
{   int base=R[s];                  //以表首元素为基准,base存放基准关键字
    int i=s,j=t+1;
    while (i<j)                     //i<j时循环
    { j--;                          //先移动避免重复比较，j初始化为t+1而不是t
      while (R[j]>base && i<j)  //后→前找一个小于等于基准的R[j]
          j--;
      i++;                                  //先移动避免重复比较
      while (R[i]<=base && i<j)             //前→后找一个大于基准的R[i]
          i++;
      if (i<j)                              //i<j时交换
         swap(R[i],R[j]);                   //保证i左侧≤base，j右侧>base
    }
    swap(R[s],R[j]);                        //将基准R[s]和R[j]进行交换
    return j;
}
```

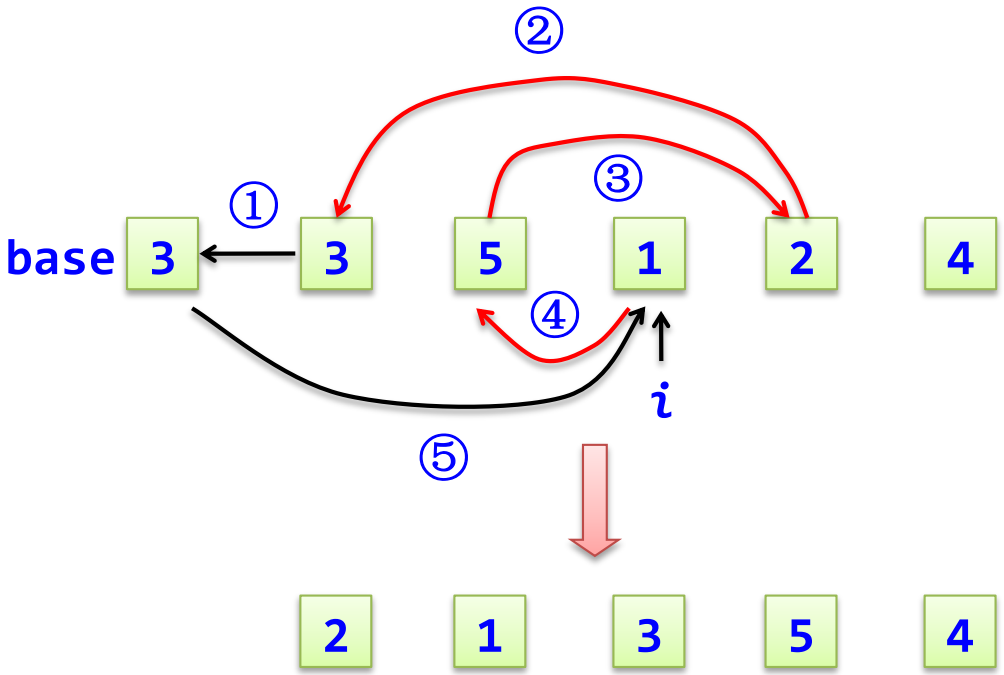
注意外while结束时不一定满足i=j，可能i>j

采用的划分算法

方法2



直到i=j，再将基准base放在R[i]



说明

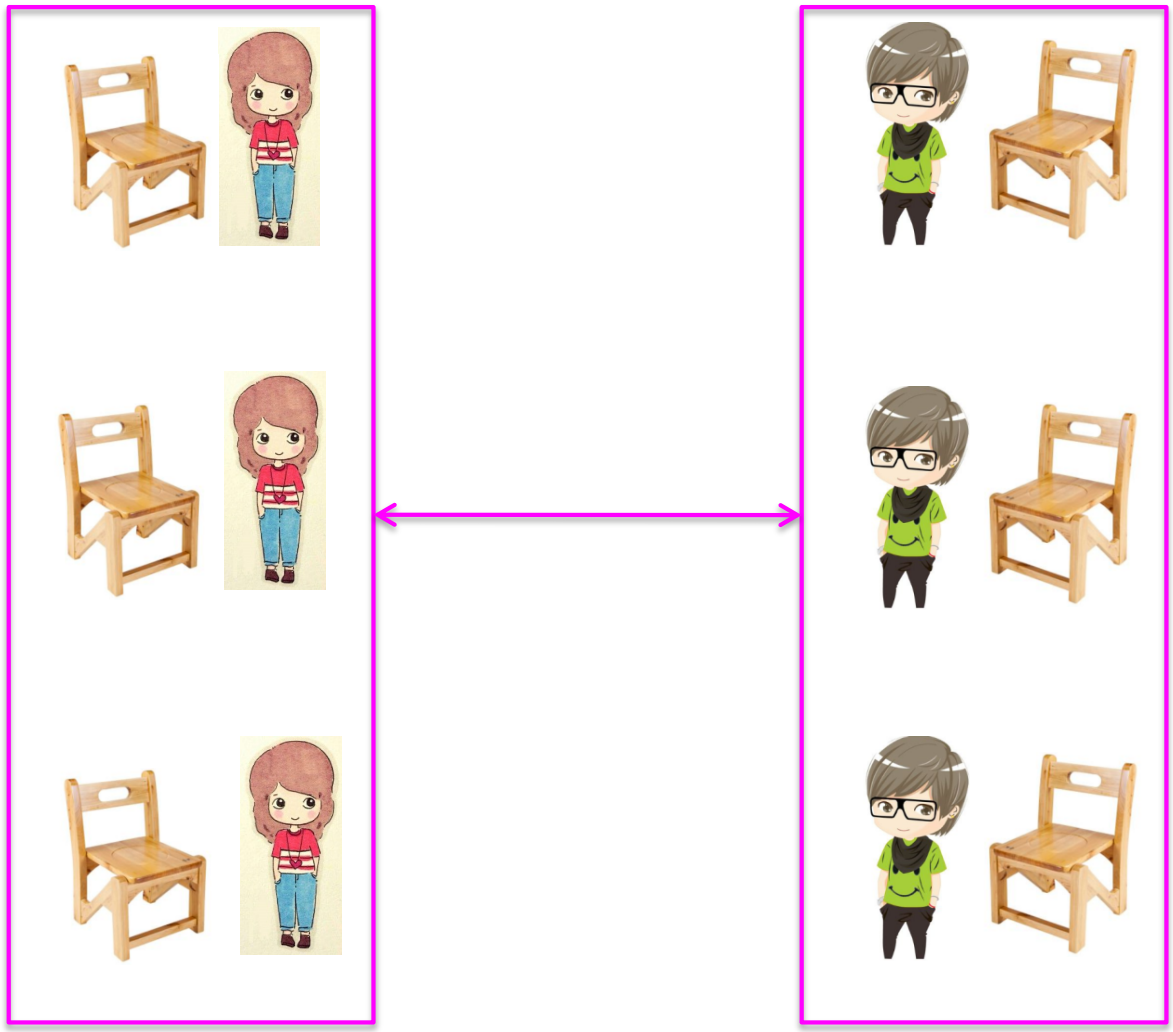
*n*个元素共比较*n*-1次

```
int Partition2(vector<int>& R,int s,int t)  //划分算法2
{  int i=s,j=t;
   int base=R[s];                          //以表首元素为基准
   while (i<j)                    //从表两端交替向中间遍历,直至i=j为止
   {  while (j>i && R[j]>base)
         j--;                              //后→前找一个小于等于基准的R[j]
      if (j>i)
      {  R[i]=R[j];                        //R[j]前移覆盖R[i]
         i++;
      }
      while (i<j && R[i]<=base)
         i++;                              //前→后找一个大于基准的R[i]
      if (i<j)
      {  R[j]=R[i];                        //R[i]后移覆盖R[j]
         j--;
      }
   }
   R[i]=base;                              //基准归位
   return i;                               //返回归位的位置
}
```

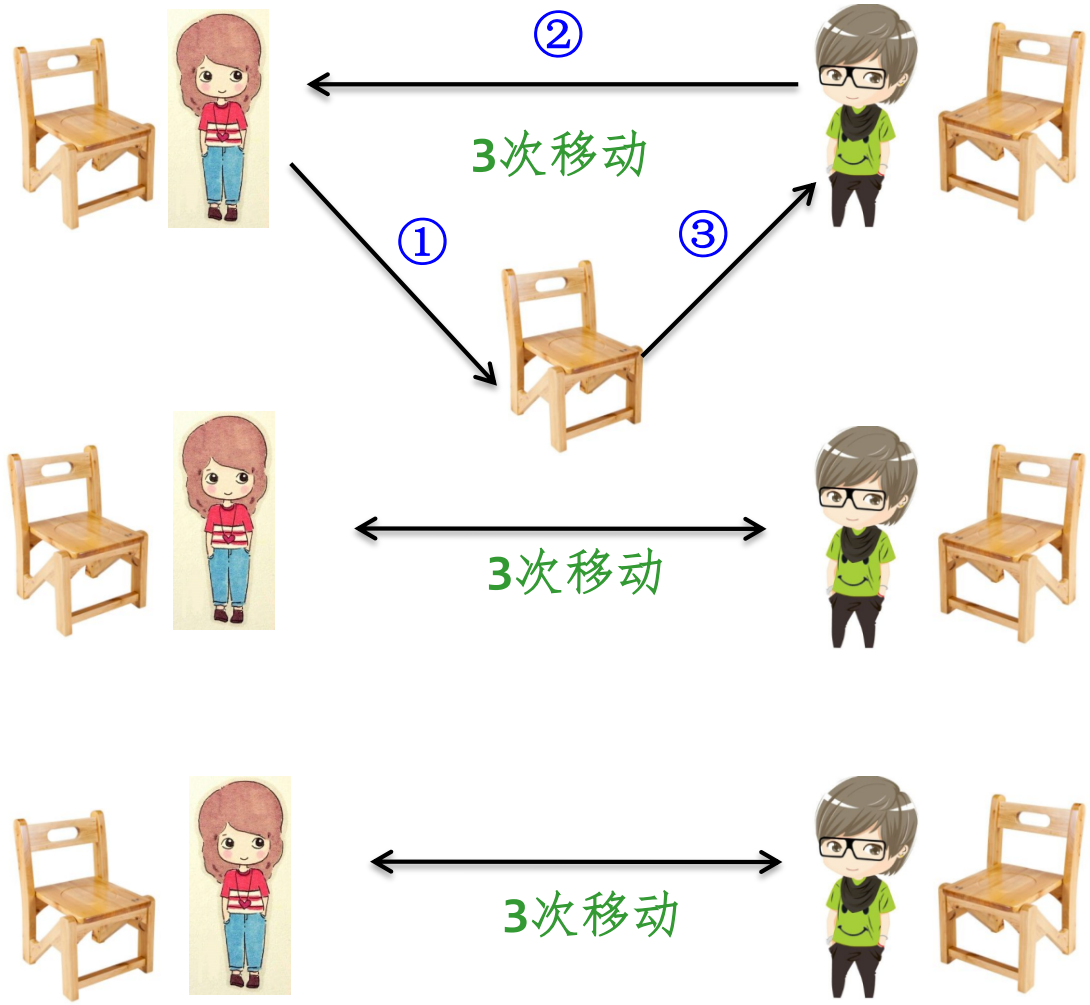
国内教材采用的划分方法

Partition1 和Partition2 在算法复杂度上有区别吗？

两个组交换位置

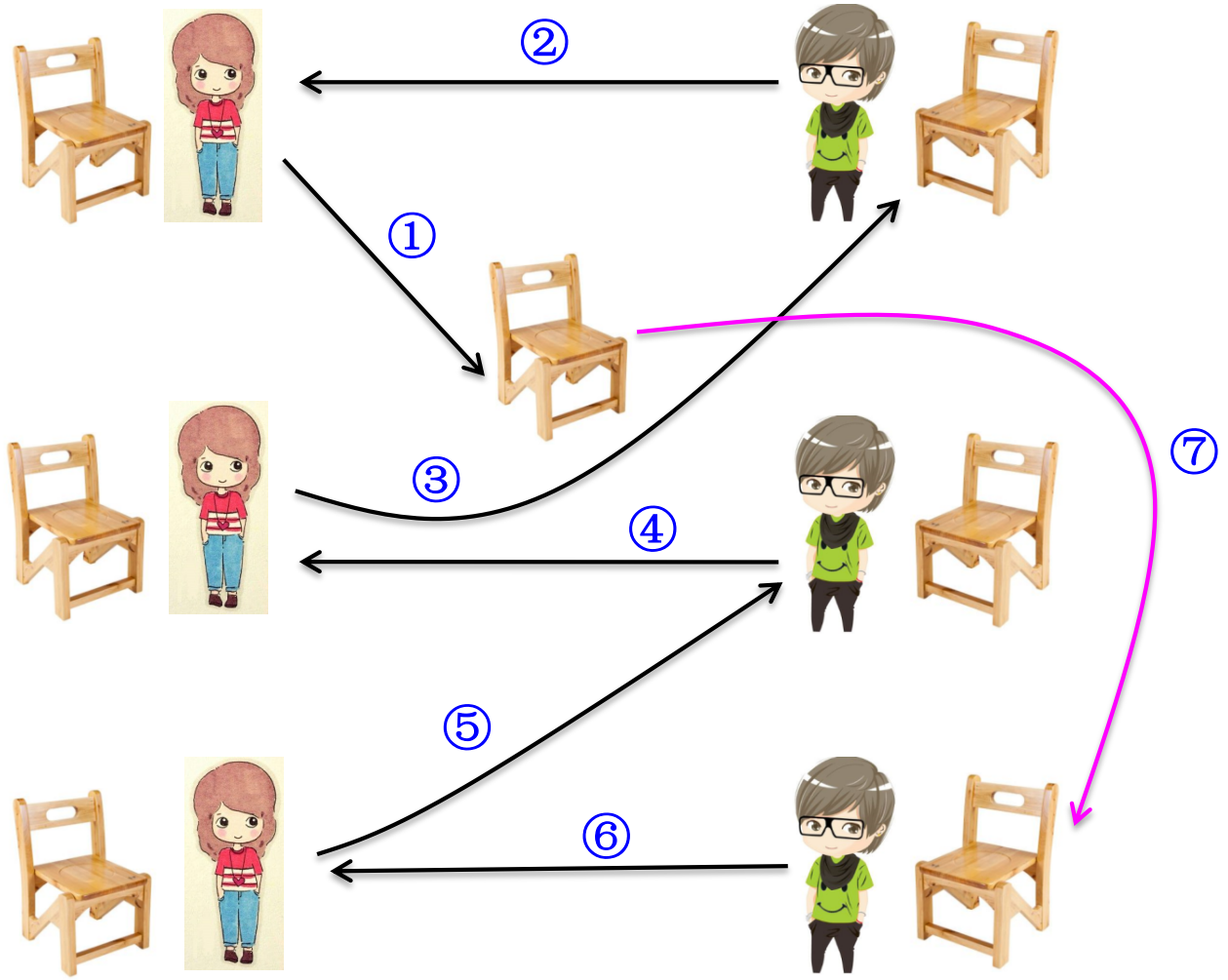


两个组交换位置



共9次移动

两个组交换位置



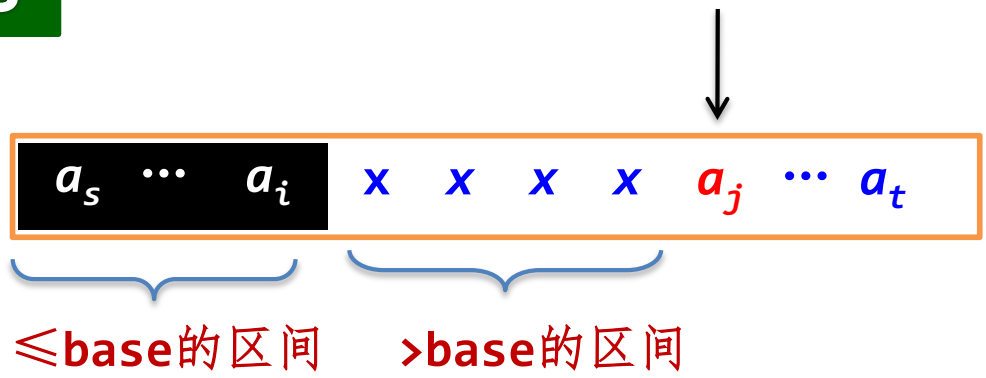
共7次移动

为什么Partition2比Partition1好？

减少了元素移动次数！

- 这里是根据base分为两组，两组的元素交换
- 两组的元素个数相同

方法3

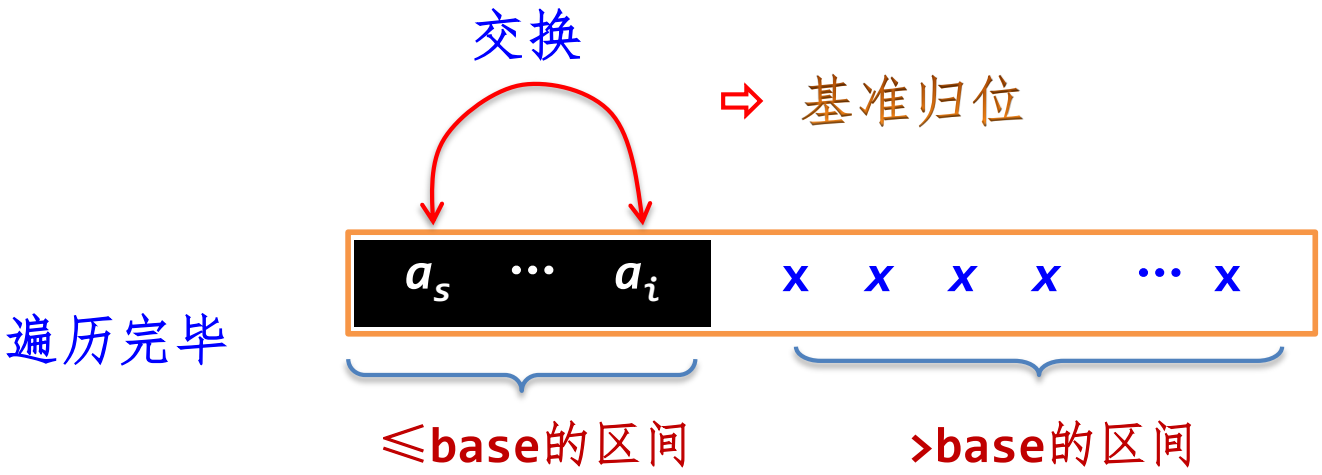


初始时，“≤base的区间”含a_s ⇨ i=s，j从s+1开始遍历，“>base的区间”是a[i+1..j-1]

- 若a[j]>base，跳过，j++。
- 若a[j]<=base，i增加1扩大“≤base的区间”，将a[j]与a[i]交换，扩大“>base的区间”，j++。

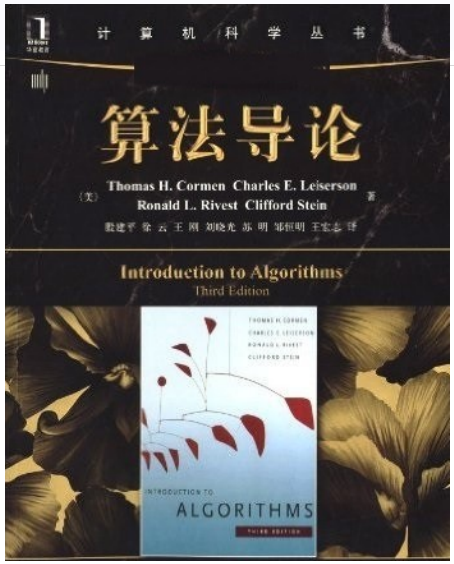
说明

n个元素共比较n-1次



《算法导论》的思路（以尾元素为基准）

```
int Partition3(vector<int>& R,int s,int t) //划分算法3
{  int i=s,j=s+1;
   int base=R[s];                          //以表首元素为基准
   while (j<=t)                            //j从s+1开始遍历其他元素
   {  if (R[j]<=base)                      //找到小于等于基准的元素R[j]
      {  i++;                              //扩大小于等于base的元素区间
         if (i!=j)
            swap(R[i],R[j]);               //将R[i]与R[j]交换
      }
      j++;                                 //继续扫描
   }
   swap(R[s],R[i]);                        //将基准R[s]和R[i]进行交换
   return i;
}
```



采用的划分算法（改为以区间尾元素为基准）

2）排序算法设计

$f(R, s, t)$ ≡ 不做任何事情　　当$R[s..t]$为空或者仅有一个元素时

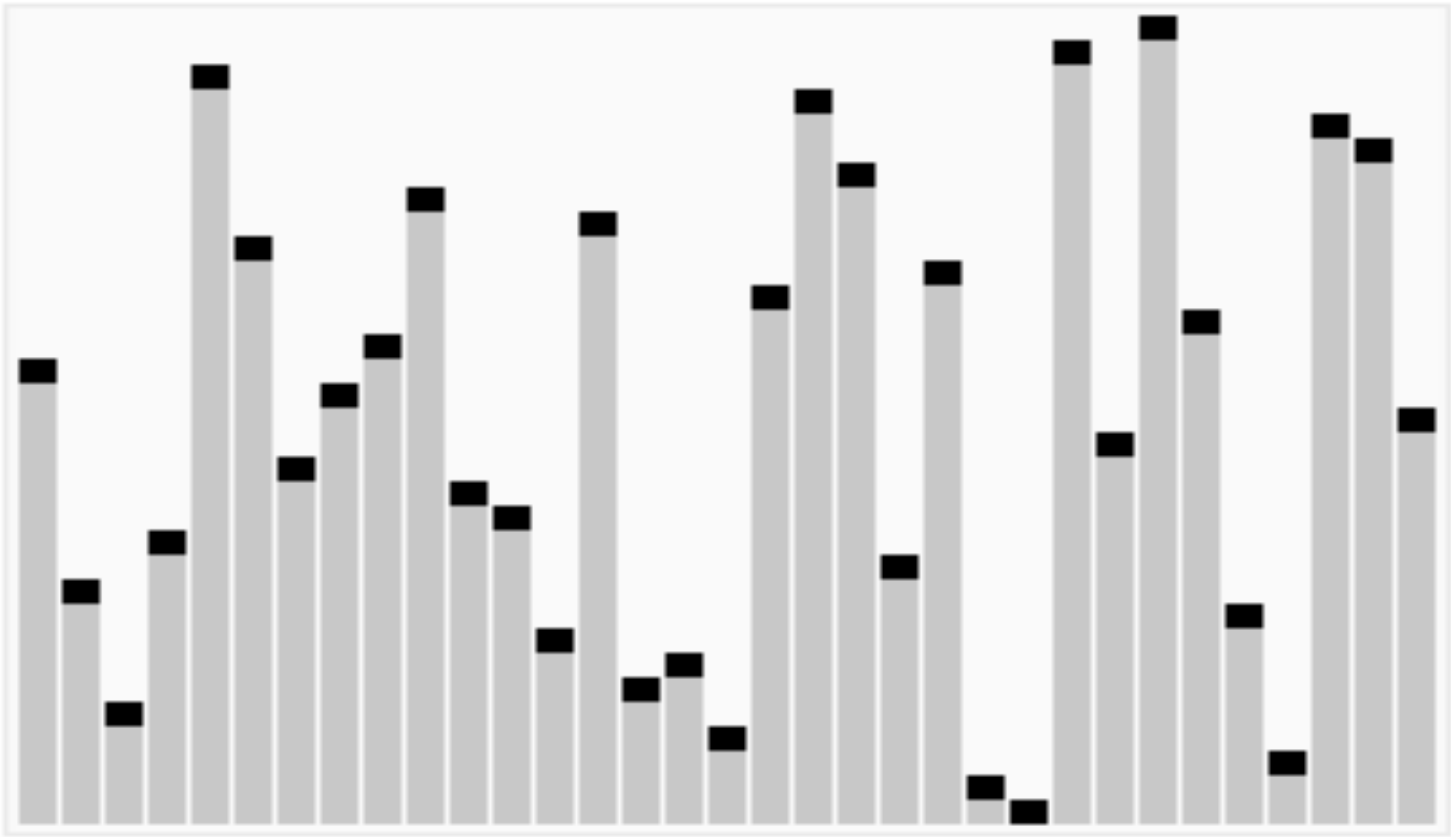
$f(R, s, t)$ ≡ 划分后基准位置为i;　　其他情况

$f(R, s, i-1)$;

$f(R, i+1, t)$;

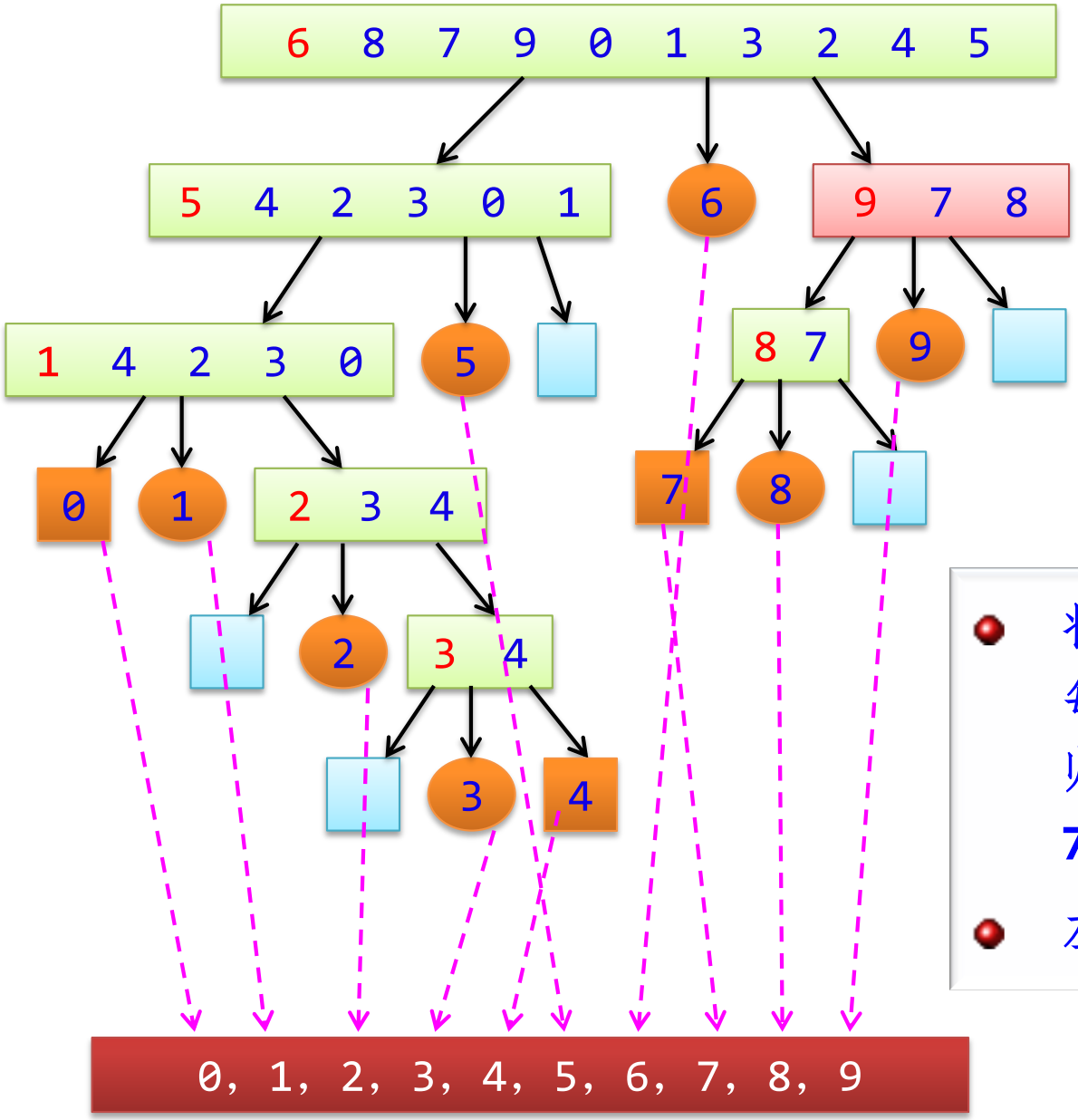
```
void _QuickSort(vector<int>& R,int s,int t)  //对R[s..t]快速排序
{  if (s<t)                                  //表中至少存在两个元素的情况
   {  int i=Partition3(R,s,t);               //可用3种划分算法任意一种
      _QuickSort(R,s,i-1);                   //对左子表递归排序
      _QuickSort(R,i+1,t);                   //对右子表递归排序
   }
}

void QuickSort(vector<int>& R,int n)  //快速排序
{
   _QuickSort(R,0,n-1);
}
```

《算法导论》的动画（以尾元素为基准）



【例10.4】 设待排序的表有10个元素，其关键字分别为（6，8，7，9，0，1，3，2，4，5）。说明采用快速排序方法进行排序的过程。

快速排序过程 ⇨ 递归树

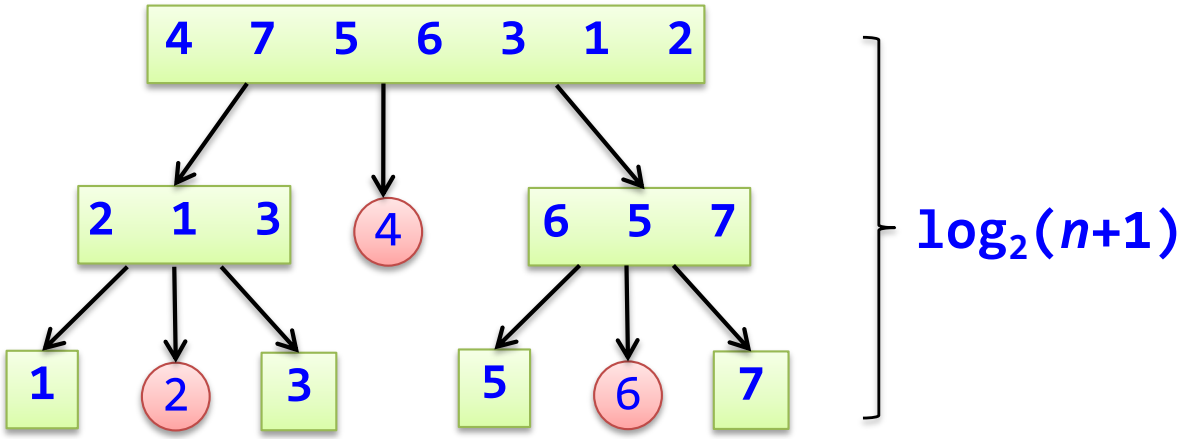


- 将递归树看成一颗3叉树，每个分支结点对应一次递归调用。这里递归次数：**7**
- 左右分区处理的顺序无关

3．算法分析

1）最好情况分析

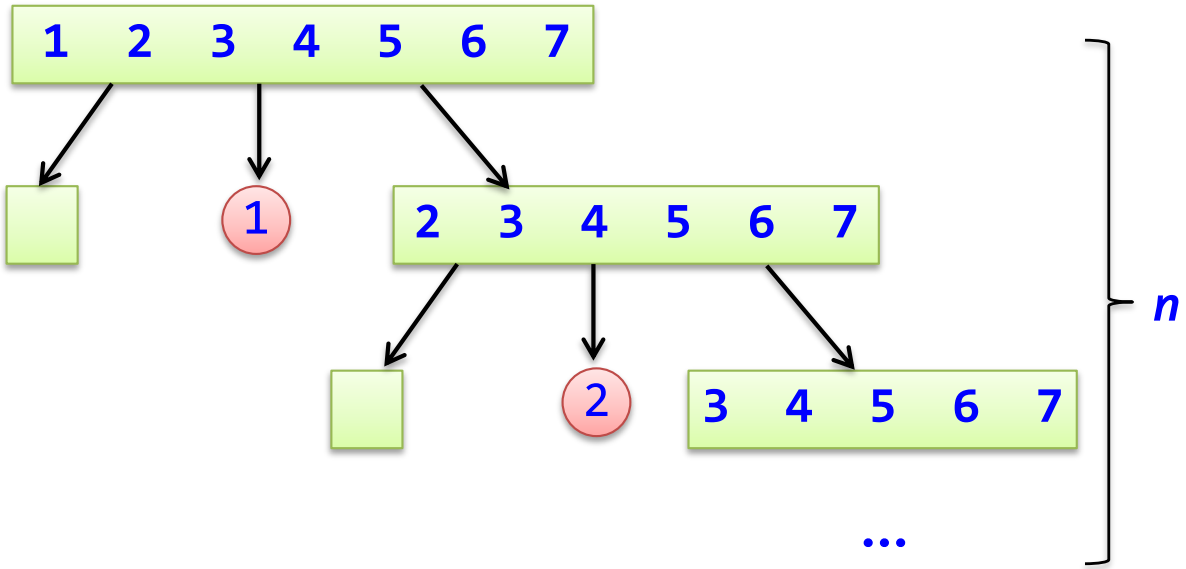
如果初始数据序列随机分布，使得每次划分恰好分为两个长度相同的子表，此时递归树的高度最小，性能最好。



最好情况下的时间复杂度为$O(n\log_2 n)$。

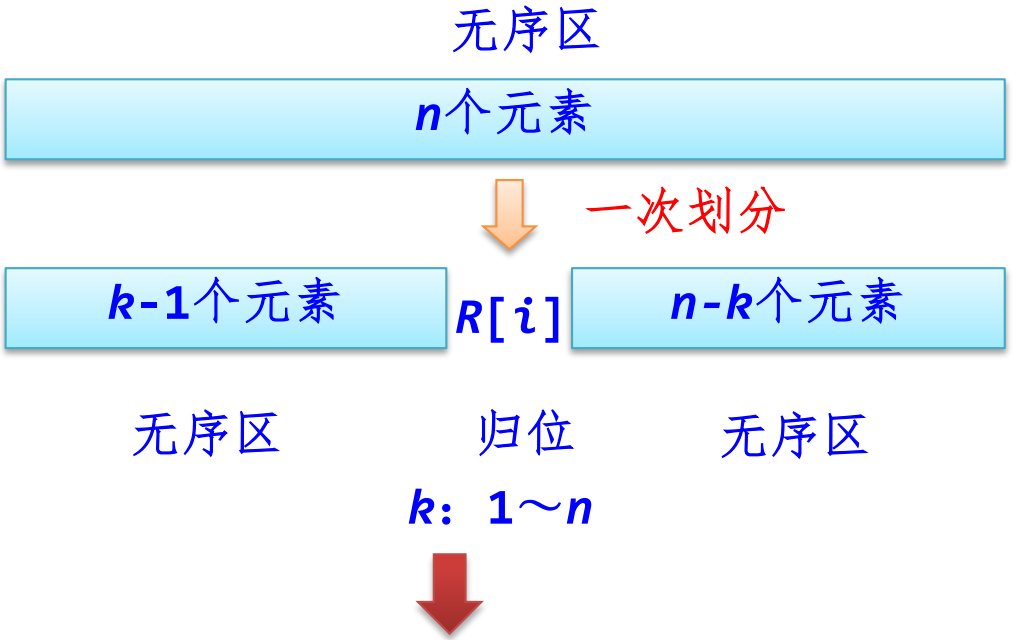
2）最坏情况分析

如果初始数据序列正序或者反序，使得每次划分的两个子表中一个为空一个长度为*n*-1，此时递归树的高度最高，性能最差。



最坏情况下的时间复杂度为$O(n^2)$。

3）平均情况分析



$$T_{avg}(n)=O(n)+\frac{1}{n}\sum_{k=1}^{n}(T_{avg}(k-1)+T_{avg}(n-k))$$

$$=cn+\frac{1}{n}\sum_{k=1}^{n}(T_{avg}(k-1)+T_{avg}(n-k))$$

$$=\cdots$$

$$=O(n\log_2 n)$$

平均情况的时间复杂度为$O(n\log_2 n)$。

【例10.5】设计一个以排序序列的中间位置元素为基准的快速排序算法。

- 对于排序序列R[s..t]，当其中个数大于1时，其中间位置mid=(s+t)/2，将首元素R[s]与R[mid]交换。
- 再采用以首元素为基准的一般快速排序方法。

```
void QuickSort1(vector<int>& R,int s,int t)
//以排序序列的中间位置元素为基准的快速排序
{  if (s<t)                                  //表中至少存在两个元素的情况
   {  int mid=(s+t)/2;
      swap(R[s],R[mid]);                     //R[s]与R[mid]交换
      int i=Partition2(R,s,t);               //可用3种划分算法中的任意一种
      QuickSort1(R,s,i-1);                   //对左子表递归排序
      QuickSort1(R,i+1,t);                   //对右子表递归排序
   }
}
```

扩展

2016年全国计算机学科专业考研题

已知由n（$n\geqslant 2$）个正整数构成的集合$A=\{a_k\}$（$0\leqslant k<n$），将其划分为两个不相交的子集A_1和A_2，元素个数分别是n_1和n_2，A_1和A_2中元素之和分别为S_1和S_2。设计一个尽可能高效的划分算法，满足$|n_1-n_2|$最小且$|S_1-S_2|$最大。要求：

（1）给出算法的基本设计思想。

（2）根据设计思想，采用C、C++描述算法，关键之处给出注释。

（3）说明你所设计算法的时间复杂度和空间复杂度。

将最小的$\lfloor n/2 \rfloor$个元素放在A_1中，其他放在A_2中

查找第$n/2$小的元素

递归快速排序

```
int Solution(vector<int> a,int n)    //求解算法
{  int low=0,high=n-1;
   bool flag=true;
   while (flag)
   {  i=Partition(a,low,high);       //可用前面3种划分算法中任意一种
      if (i==n/2-1)                  //基准a[i]为第n/2的元素
         flag=false;
      else if (i<n/2-1)              //在右区间查找
         low=i+1;
      else
         high=i-1;                   //在左区间查找
   }
   int s1=0,s2=0;
   for (int i=0;i<n/2;i++)
       s1+=a[i];
   for (int j=n/2;j<n;j++)
       s2+=a[j];
    return s2-s1;
}
```

时间复杂度为$O(n)$

15. 【2010 统考真题】采用递归方式对顺序表进行快速排序。下列关于递归次数的叙述中，正确的是（ ）。

A. 递归次数与初始数据的排列次序无关

B. 每次划分后，先处理较长的分区可以减少递归次数

C. 每次划分后，先处理较短的分区可以减少递归次数

D. 递归次数与每次划分后得到的分区的处理顺序无关

16.【2011 统考真题】为实现快速排序算法，待排序序列宜采用的存储方式是（ ）。

A. 顺序存储　　B. 散列存储　　C. 链式存储　　D. 索引存储

17.【2014 统考真题】下列选项中，不可能是快速排序第二趟排序结果的是（ ）。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9

C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

18. 【2019 统考真题】排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一“趟”。下列序列中，不可能是快速排序第二趟结果的是（ ）。

A. 5, 2, 16, 12, 28, 60, 32, 72　　B. 2, 16, 5, 28, 12, 60, 32, 72

C. 2, 12, 16, 5, 28, 32, 72, 60　　D. 5, 2, 12, 28, 16, 32, 72, 60

19. 【2023 统考真题】使用快速排序算法对数据进行升序排序，若经过一次划分后得到的数据序列是 68, 11, 70, 23, 80, 77, 48, 81, 93, 88，则该次划分的枢轴是（　）。

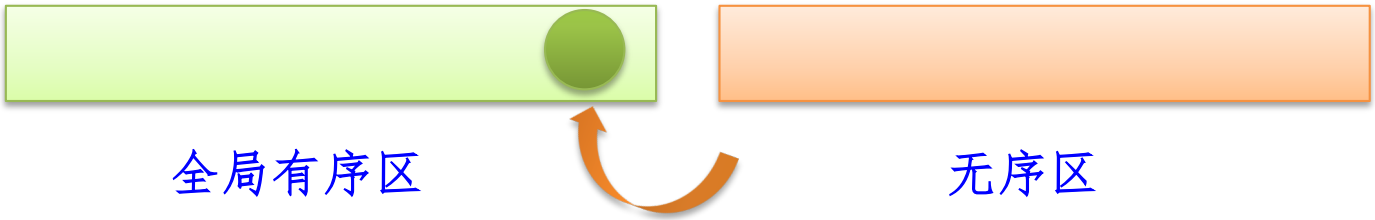
A. 11　　B. 70　　C. 80　　D. 81

各种内排序方法的比较和选择

排序方法	时间复杂度			空间复杂度	稳定性
	平均情况	最坏情况	最好情况		
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(\log_2 n)$	不稳定

10.4 选择排序

基本思路



主要的选择排序方法：

（1）简单选择排序

（2）堆排序

10.4.1 简单选择排序

1. 排序思路

从一个无序区中选出最小的元素，最简单方法是逐个进行元素比较，例如，从无序区*R*[*i*..*n*-1]中选出最小元素*R*[minj]。

```
minj=i;                          //minj先置为区间中的首元素序号
for (int j=i+1;j<n;j++)          //从R[i..n-1]中选最小元素的R[minj]
   if (R[j]<R[minj])             //与区间中其他元素比较
      minj=j;
```

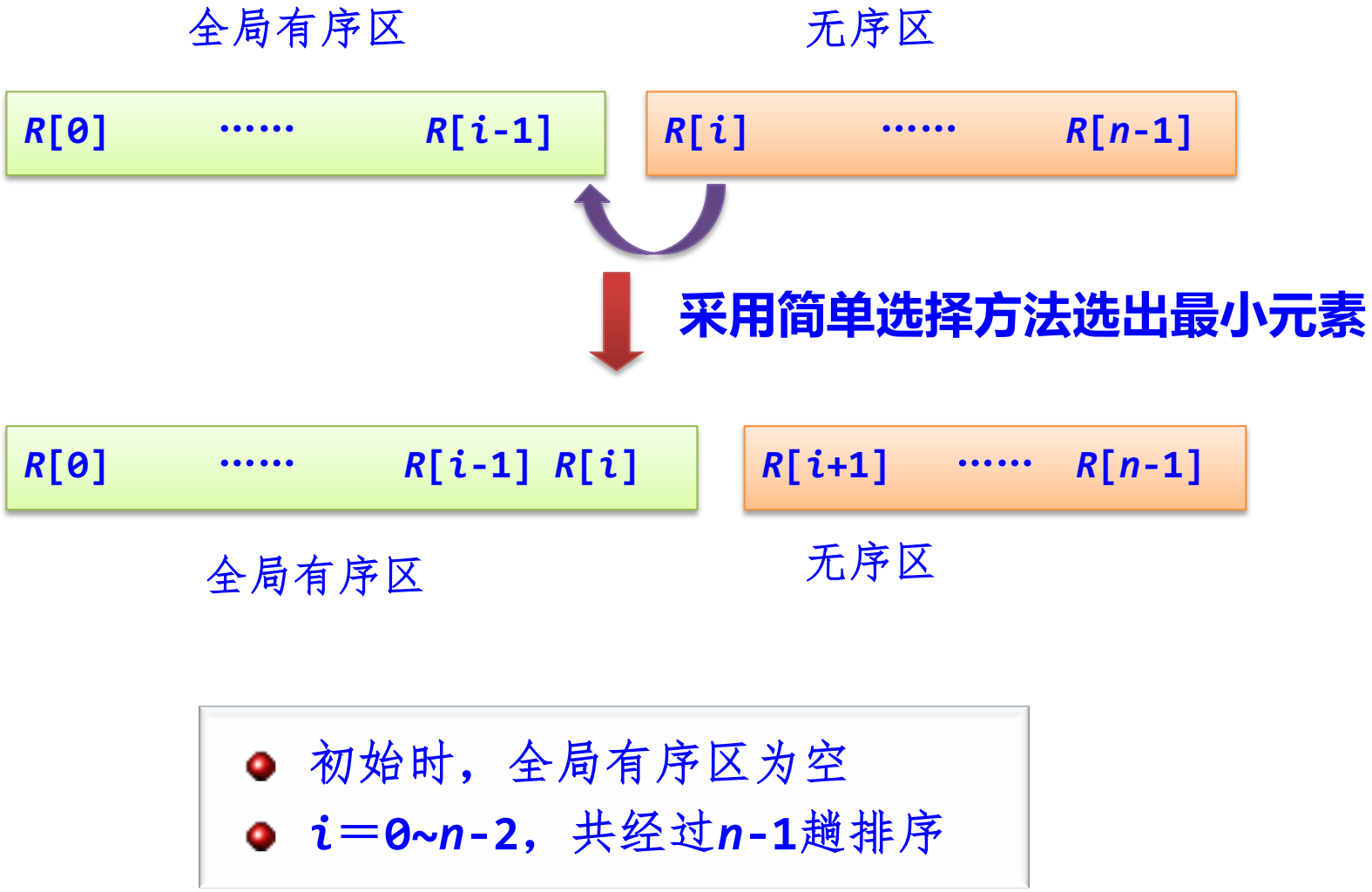
简单选择

示例

有2^n个整数，找到其中最大整数需要比较次数最少是（　　）次。

A. n　　B. $\log_2 n$　　C. n^2

D. 2^n　　E. 2^n-1　　F. $n-1$



- 初始时，全局有序区为空
- $i=0\sim n-2$，共经过$n-1$趟排序

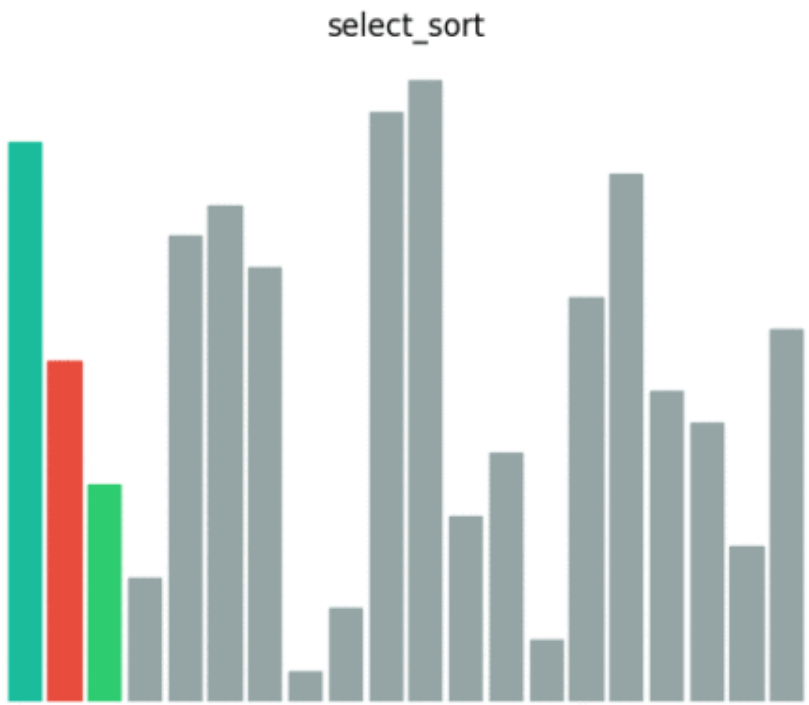
【例10.6】 设待排序的表有10个元素，其关键字分别为（6，8，7，9，0，1，3，2，4，5）。说明采用简单选择排序方法进行排序的过程。

初始关键字	[]6	8	7	9	0	1	3	2	4	5
i=0的结果：	[0]	8	7	9	6	1	3	2	4	5
i=1的结果：	[0	1]	7	9	6	8	3	2	4	5
i=2的结果：	[0	1	2]	9	6	8	3	7	4	5
i=3的结果：	[0	1	2	3]	6	8	9	7	4	5
i=4的结果：	[0	1	2	3	4]	8	9	7	6	5
i=5的结果：	[0	1	2	3	4	5]	9	7	6	8
i=6的结果：	[0	1	2	3	4	5	6]	7	9	8
i=7的结果：	[0	1	2	3	4	5	6	7]	9	8
i=8的结果：	[0	1	2	3	4	5	6	7	8]	9

2．排序算法

```
void SelectSort(vector<int>& R,int n)        //简单选择排序
{  for (int i=0;i<n-1;i++)    //做第i趟排序
   {  int minj=i;
      for (int j=i+1;j<n;j++)//在当前无序区R[i..n-1]中选最小元素R[minj]
         if (R[j]<R[minj])
            minj=j;           //minj记下目前找到的最小元素的位置
      if (minj!=i)            //若R[minj]不是无序区首元素
         swap(R[i],R[minj]); //交换R[i]和R[minj]
   }
}
```

简单选择排序



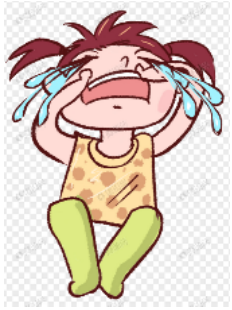
3．算法分析

无论初始数据序列的状态如何，在第**i**趟排序从无序区**R[i..n-1]**（含**n-i**个元素）中选出最小元素时，内**for**循环需做***n*-*i*-1**次比较，因此，总的比较次数为

$$C(n)=\sum_{i=0}^{n-2}(n-i-1)=\frac{n(n-1)}{2}=O(n^2)$$

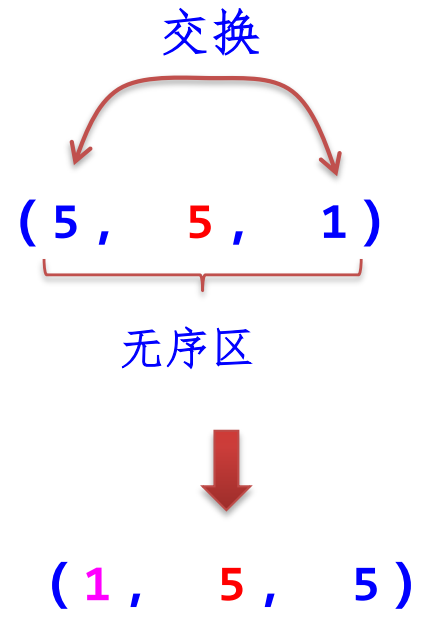
元素的移动次数

- 当初始数据序列正序时，移动次数为0。
- 反序时每趟排序均要执行交换操作，此时总的移动次数为最大值3(*n*-1)。



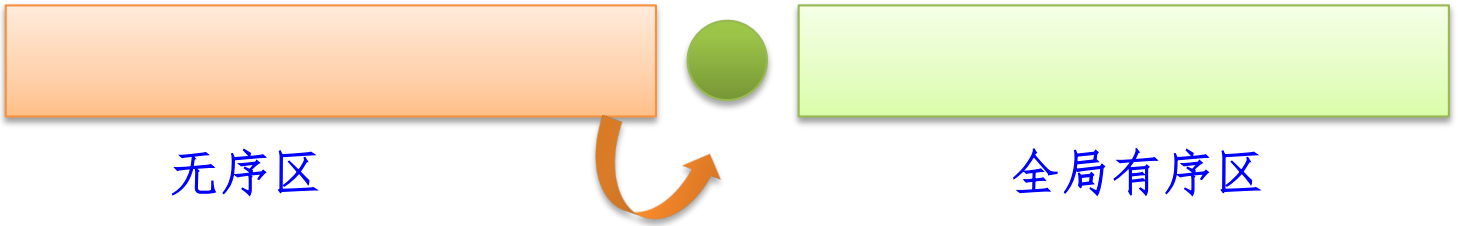
最好、最坏和平均情况的时间复杂度均为$O(n^2)$。

是一种不稳定的排序方法



10.4.2 堆排序

1．排序思路



选出最大元素$R[k]$

采用堆方法选出最大元素：堆排序算法

堆的定义

一个序列$R[0..n-1]$，关键字分别为k_0、k_1、…、k_{n-1}。

该序列满足如下性质（简称为堆性质）：

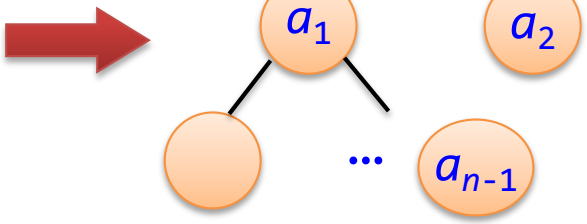
❶ $k_i \leqslant k_{2i+1}$ 且 $k_i \leqslant k_{2i+2}$

或 ❷ $k_i \geqslant k_{2i+1}$ 且 $k_i \geqslant k_{2i+2}$ （$0 \leqslant i \leqslant \lfloor n/2 \rfloor - 1$）

满足第❶种情况的堆称为小根堆，满足第❷种情况的堆称为大根堆。下面讨论的堆是大根堆。

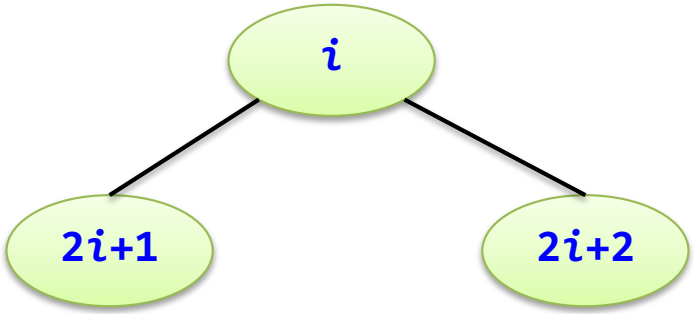
❶ 将序列$a_0\ a_1\ \cdots\ a_{n-1}$看成是一颗完全二叉树

$a_0\ a_1\ \cdots\ a_{n-1}$



完全二叉树

层序编号方式：



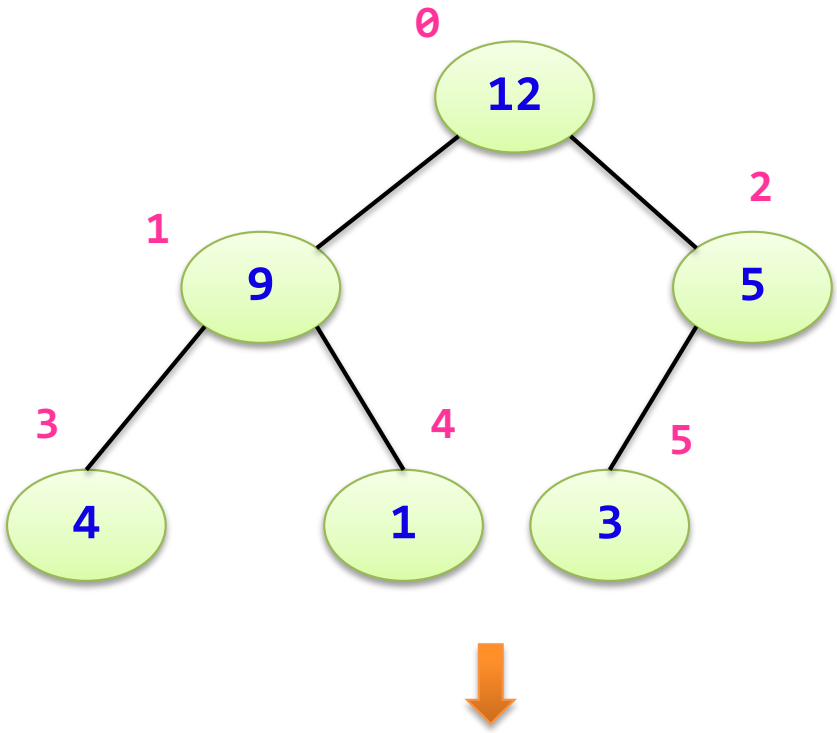
左孩子　　右孩子

- 大根堆：对应的完全二叉树中，任意一个结点的关键字都大于或等于它的孩子结点的关键字。
- 最小关键字的元素一定是某个叶子结点！！！

❷ 如何判断一颗完全二叉树是否为大根堆

n=6

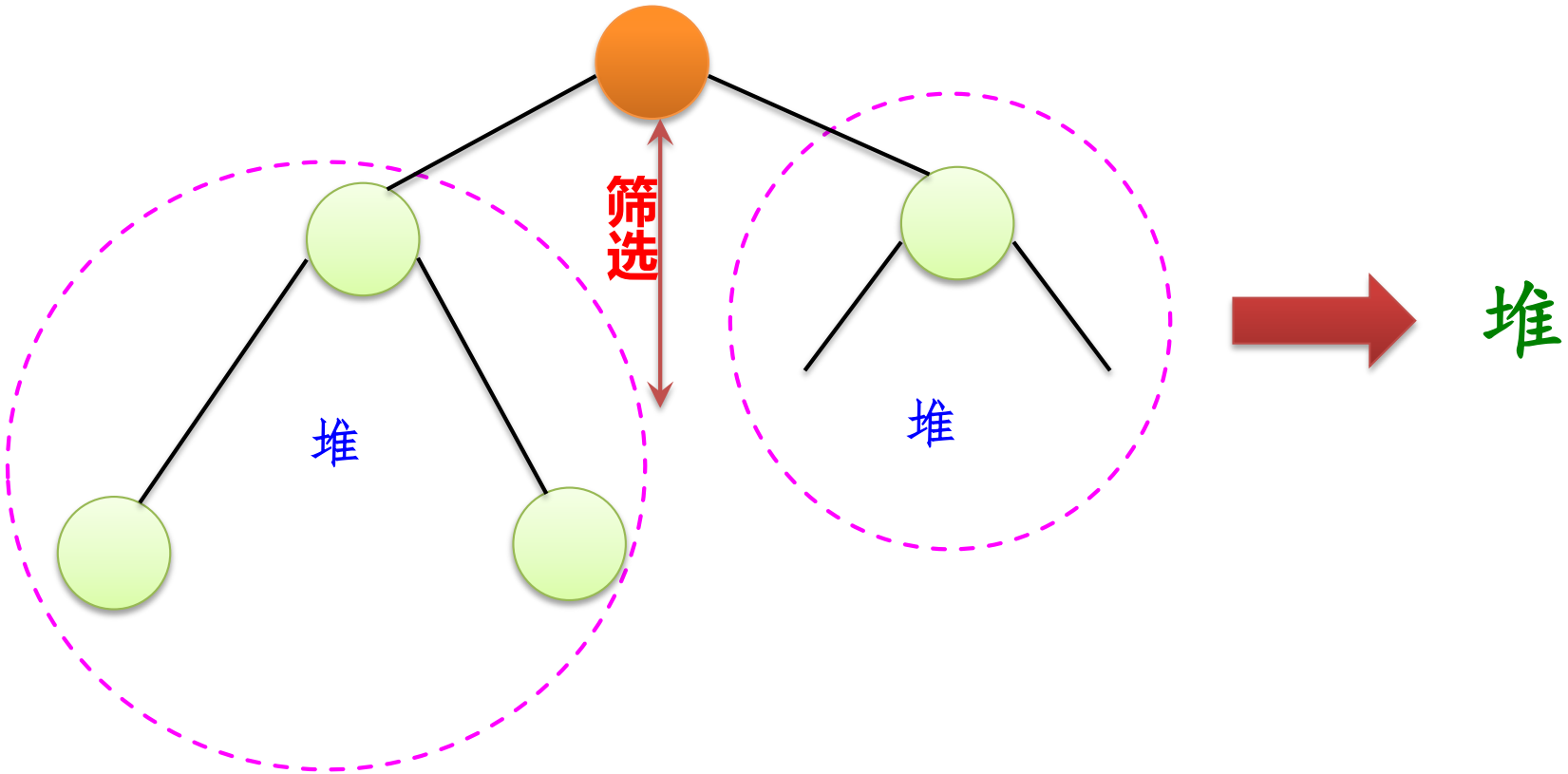
从编号为*n*/2-1=2的结点开始，逐一判断所有分支结点



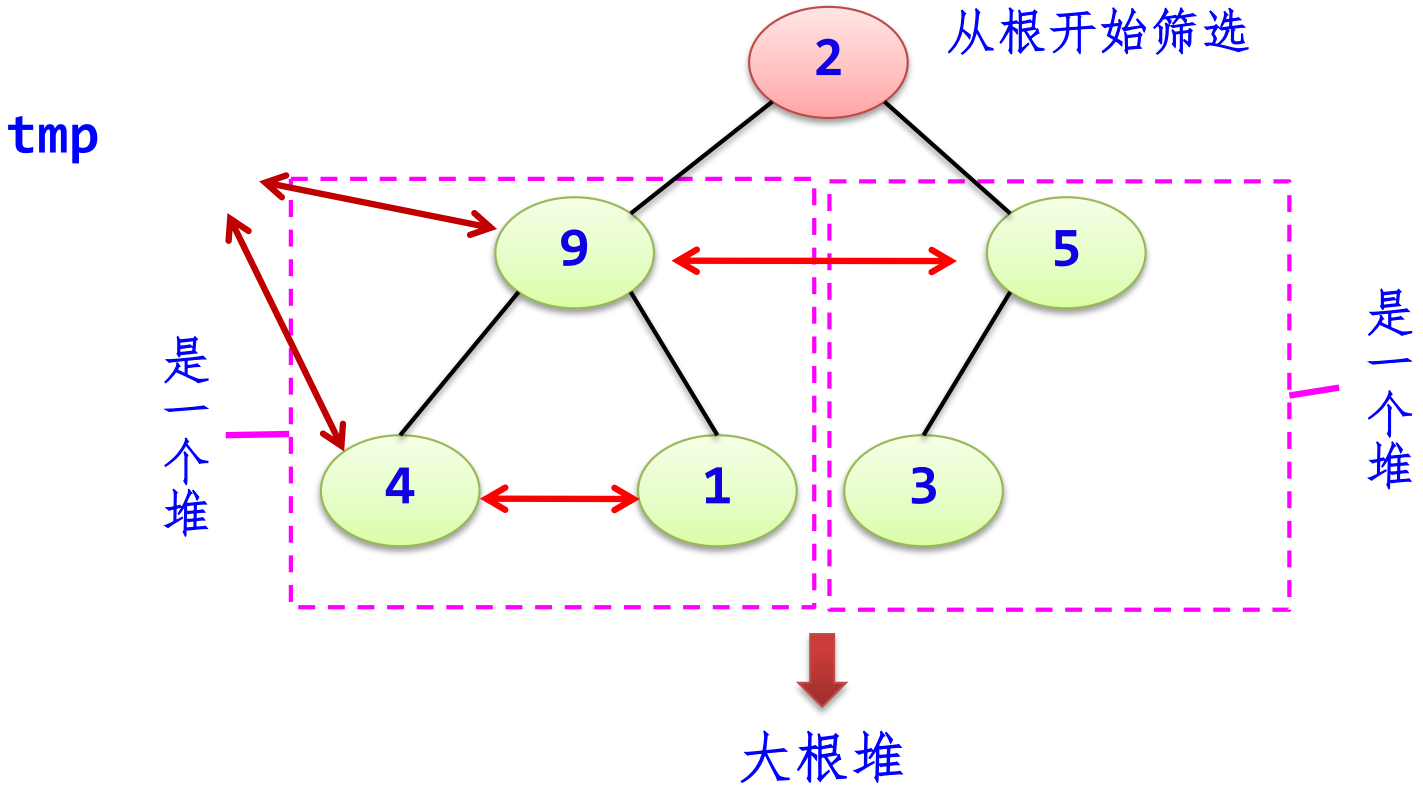
所有分支结点满足定义 ⇨ 为大根堆

2．排序算法

- 堆排序的关键是构造堆，这里采用筛选算法建堆。
- 所谓“筛选”指的是，对一棵左/右子树均为堆的完全二叉树，“调整”根结点使整个二叉树也成为一个堆。

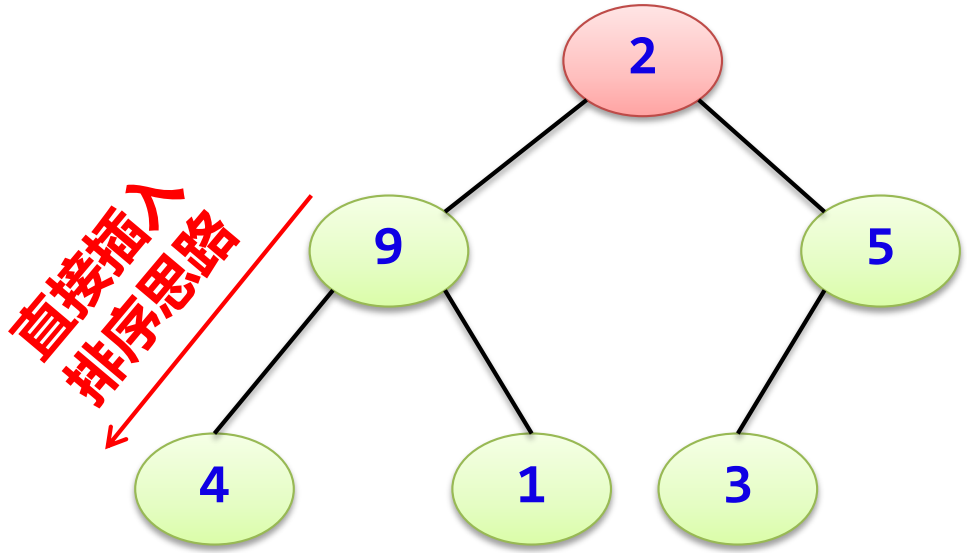


❶ 筛选：不是堆 ⇨ 堆

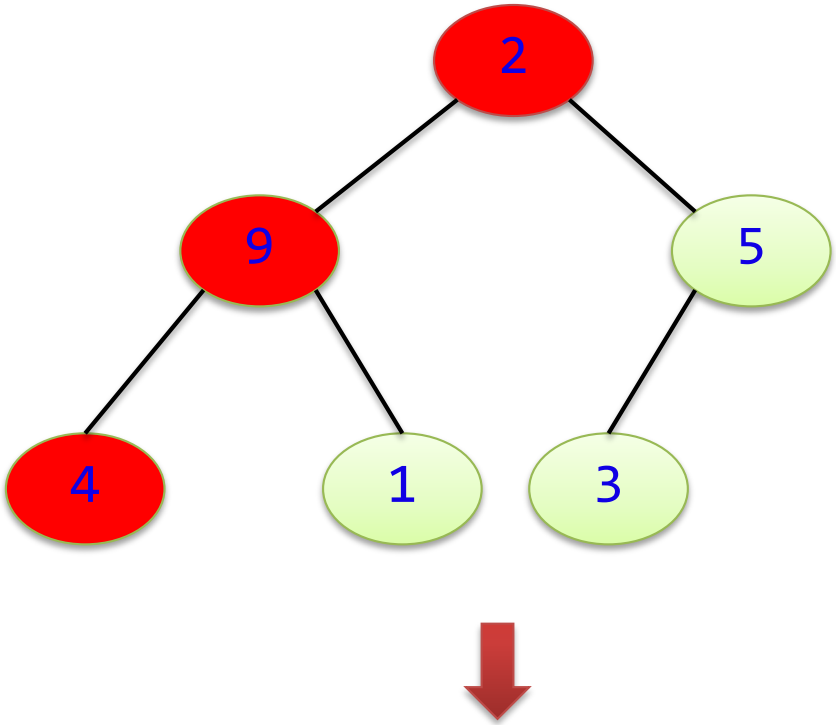


- 自顶向下筛选：从根结点R[low]开始向下依次查找较大的孩子结点，构成一个序列（2，9，4），其中除了2外其他元素的子序列恰好是递减的。
- 采用类似直接插入排序的思路使其成为一个递减序列（因为大根堆中从根到每个叶子结点的路径均构成一个递减序列）。

从根开始筛选



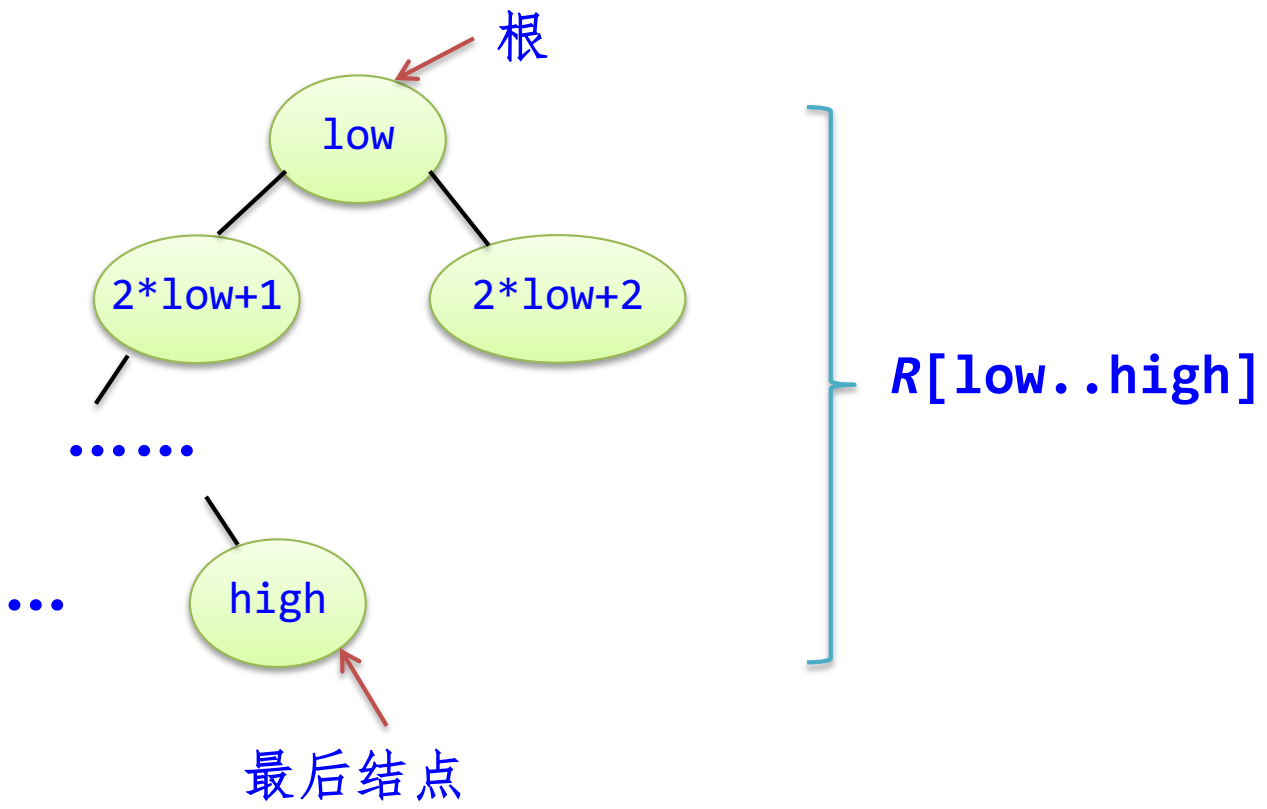
从根开始筛选



- 仅仅处理从根结点⇨某个叶子结点路径上的结点
- n个结点的完全二叉树高度为$\lceil \log_2(n+1) \rceil$
- 所有筛选的时间复杂度为$O(\log_2 n)$

向下筛选算法

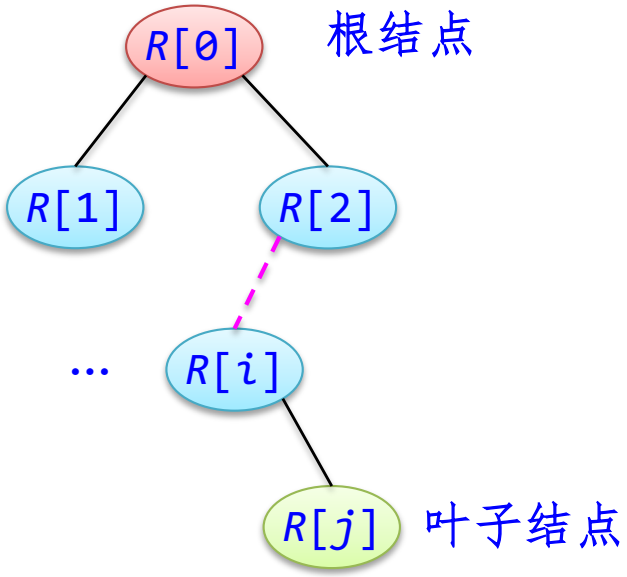
```
siftDown(RecType[] R, int low, int high): R[low..high]
```



```
void siftDown(vector<int>& R,int low,int high)
//R[low..high]的自顶向下筛选
{  int i=low;
   int j=2*i+1;                          //R[j]是R[i]的左孩子
   int tmp=R[i];                         //tmp临时保存根结点
   while (j<=high)                       //只对R[low..high]的元素进行筛选
   {  if (j<high && R[j]<R[j+1])
         j++;                            //若右孩子较大,把j指向右孩子
      if (tmp<R[j])                      //tmp的孩子较大
      {  R[i]=R[j];                      //将R[j]调整到双亲位置上
         i=j; j=2*i+1;                   //修改i和j值,以便继续向下筛选
      }
      else break;                        //若孩子较小，则筛选结束
   }
   R[i]=tmp;                             //原根结点放入最终位置
}
```

向上筛选算法

```
void siftUp(vector<int>& R,int j)   //自底向上筛选:从叶子结点j向上筛选
{  int i=(j-1)/2;                   //i指向R[j]的双亲
   while (true)
   {  if (R[j]>R[i])                //若孩子较大
         swap(R[i],R[j]);           //交换
      if (i==0) break;              //到达根结点时结束
      j=i; i=(j-1)/2;               //继续向上调整
   }
}
```



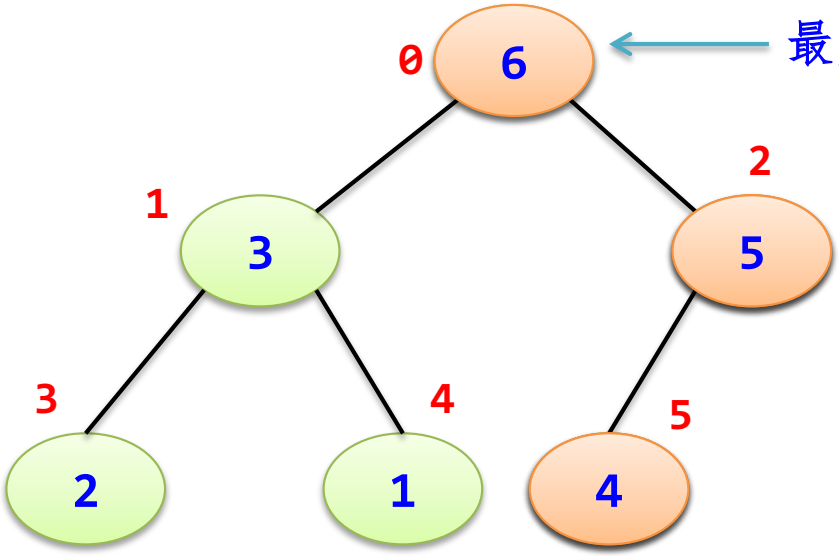
❷ 一颗完全二叉树 ⇨ 初始堆

采用向下筛选方法

例如，序列：（4，3，5，2，1，6），*n*=6

从编号为*n*/2=3的结点开始，逐一筛选

```
for (int i=n/2-1;i>=0;i--)
            //从最后一个分支结点开始循环建立初始堆
   siftDown(R,i,n-1);
        //对R[i..n-1]进行筛选
```



筛选步骤：

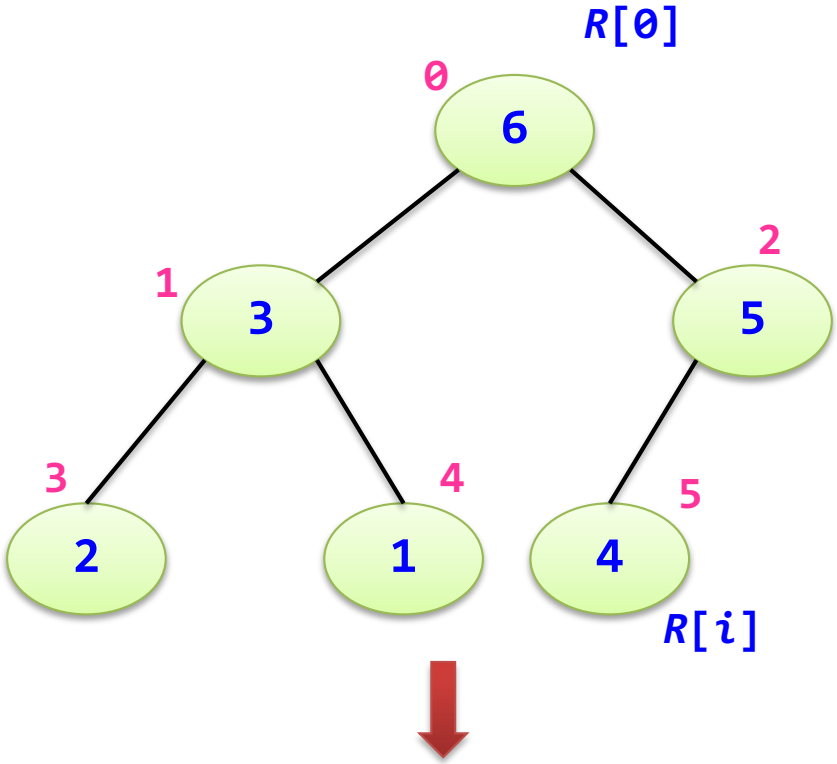
① siftDown(R,2,5)

② siftDown(R,1,5)

③ siftDown(R,0,5)

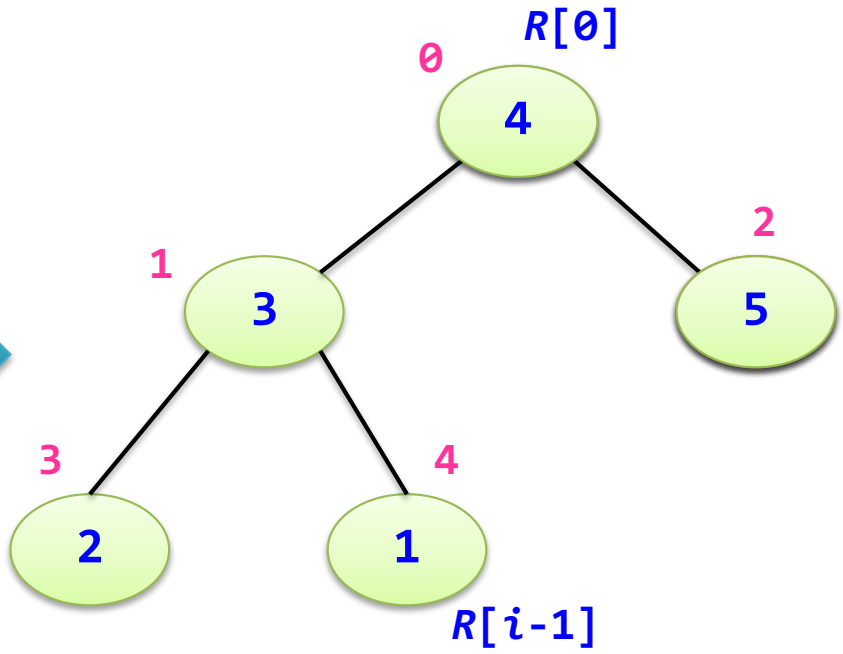
初始堆：（6，3，5，2，1，4）

❸ 最大元素归位



4，3，5，2，1，6

最大元素6归位



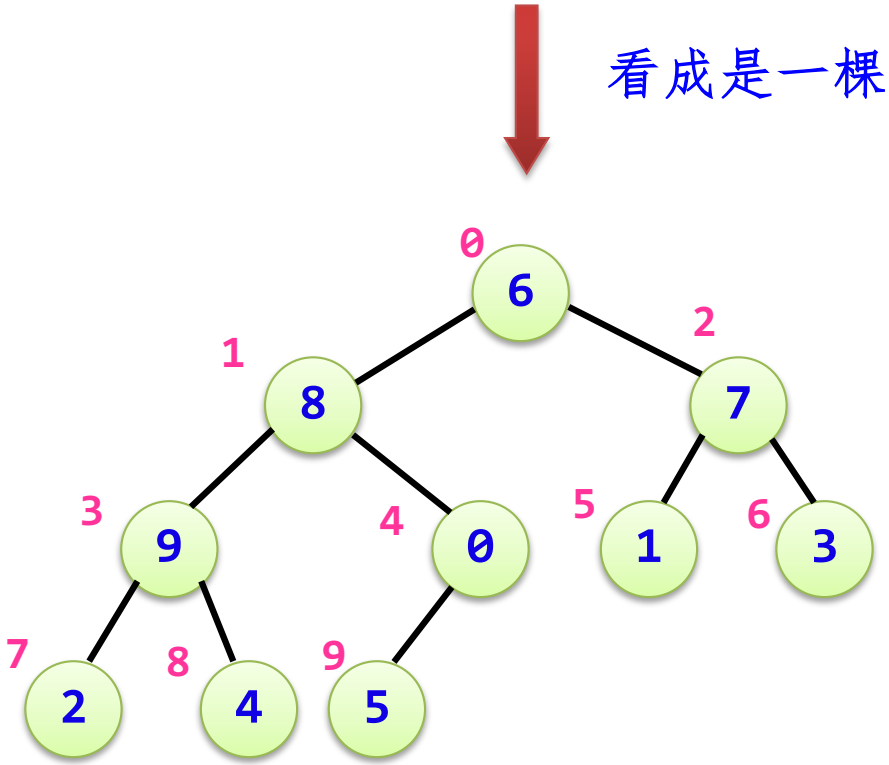
再对R[0..i-1]的记录进行筛选

堆排序算法：

```
void HeapSort(vector<int>& R,int n)  //堆排序
{  for (int i=n/2-1;i>=0;i--)//从最后一个分支结点开始循环建立初始堆
      siftDown(R,i,n-1);                  //对R[i..n-1]进行筛选

   for (int i=n-1;i>0;i--)    //进行n-1趟排序,每一趟后无序区元素个数减1
   {  swap(R[0],R[i]);        //将无序区中尾元素与R[0]交换，扩大有序区
      siftDown(R,0,i-1);                  //对无序区R[0..i-1]继续筛选
   }
}
```

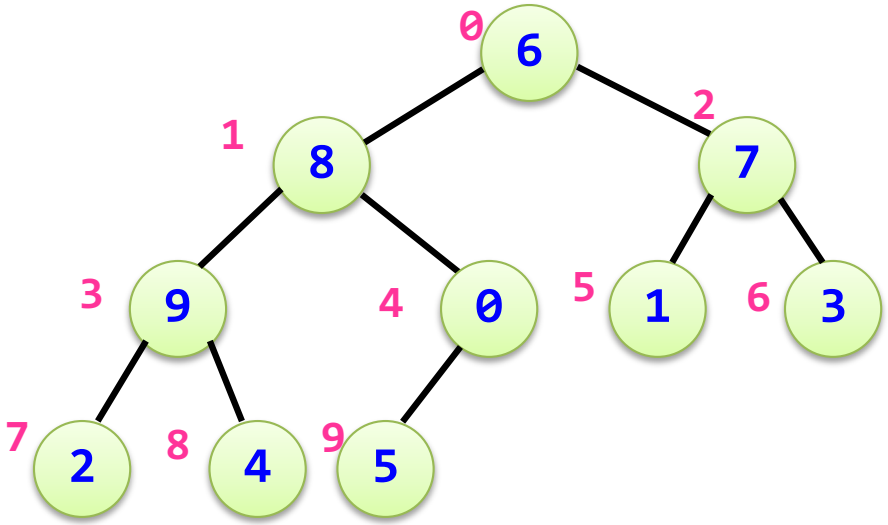
【例10.7】设待排序的表有10个记录，其关键字分别为{6，8，7，9，0，1，3，2，4，5}。说明采用堆排序方法进行排序的过程。

排序序列：6，8，7，9，0，1，3，2，4，5



```
void HeapSort(vector<int>& R,int n)
{  for (int i=n/2-1;i>=0;i--)
      siftDown(R,i,n-1);
   for (int i=n-1;i>0;i--)
   {  swap(R[0],R[i]);
      siftDown(R,0,i-1);
   }
}

void siftDown(vector<int>& R,int low,int high)
{  int i=low;
   int j=2*i+1;
   int tmp=R[i];
   while (j<=high)
   {  if (j<high && R[j]<R[j+1])
         j++;
      if (tmp<R[j])
      {  R[i]=R[j];
         i=j; j=2*i+1;
      }
      else break;
   }
   R[i]=tmp;
}
```

调整成初始大根堆：

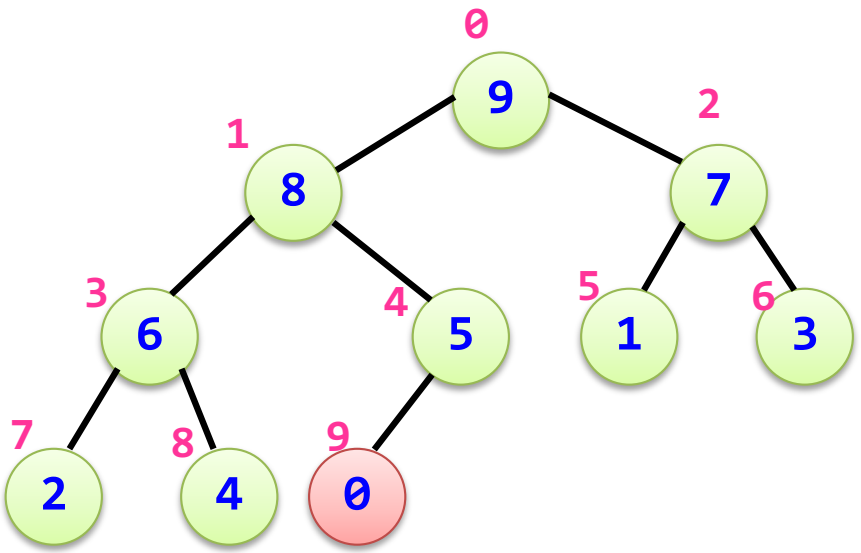


调整完毕，成为一个大根堆

9 8 7 6 5 1 3 2 4 0

```
void HeapSort(vector<int>& R,int n)
{  for (int i=n/2-1;i>=0;i--)
      siftDown(R,i,n-1);
   for (int i=n-1;i>0;i--)
   {  swap(R[0],R[i]);
      siftDown(R,0,i-1);
   }
}

void siftDown(vector<int>& R,int low,
                        int high)
{  int i=low;
   int j=2*i+1;
   int tmp=R[i];
   while (j<=high)
   {  if (j<high && R[j]<R[j+1])
         j++;
      if (tmp<R[j])
      {  R[i]=R[j];
         i=j; j=2*i+1;
      }
      else break;
   }
   R[i]=tmp;
}
```

第1趟排序

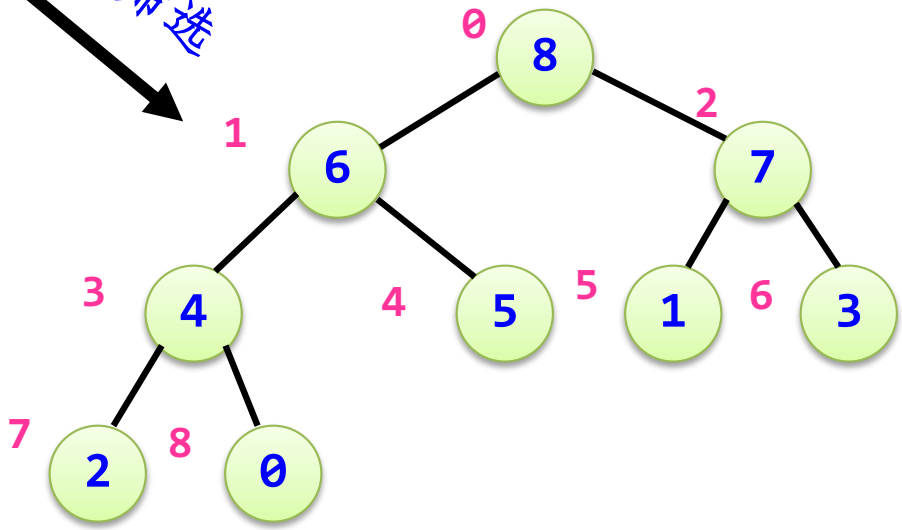


0 8 7 6 5 1 3 2 4 9

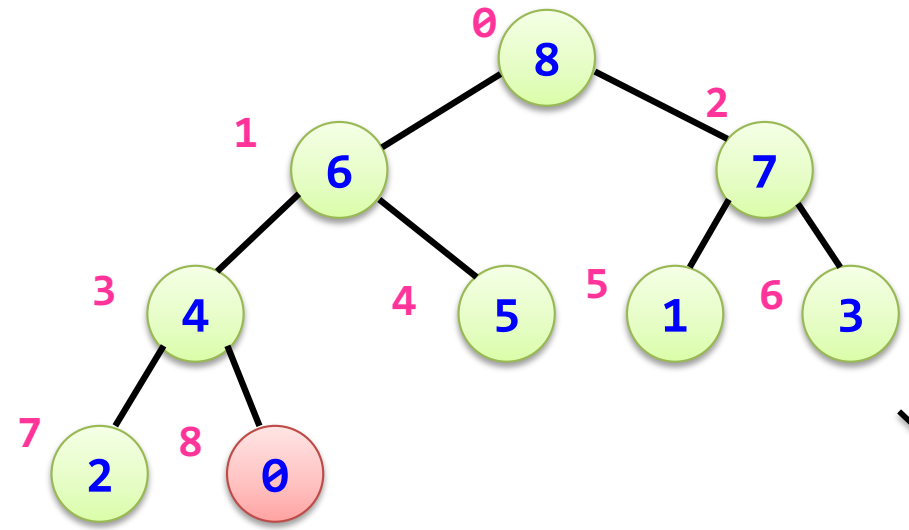
输出9（归位）

```
void HeapSort(vector<int>& R,int n)
{  for (int i=n/2-1;i>=0;i--)
       siftDown(R,i,n-1);
   for (int i=n-1;i>0;i--)
   {  swap(R[0],R[i]);
      siftDown(R,0,i-1);
   }
}
```

从根结点筛选



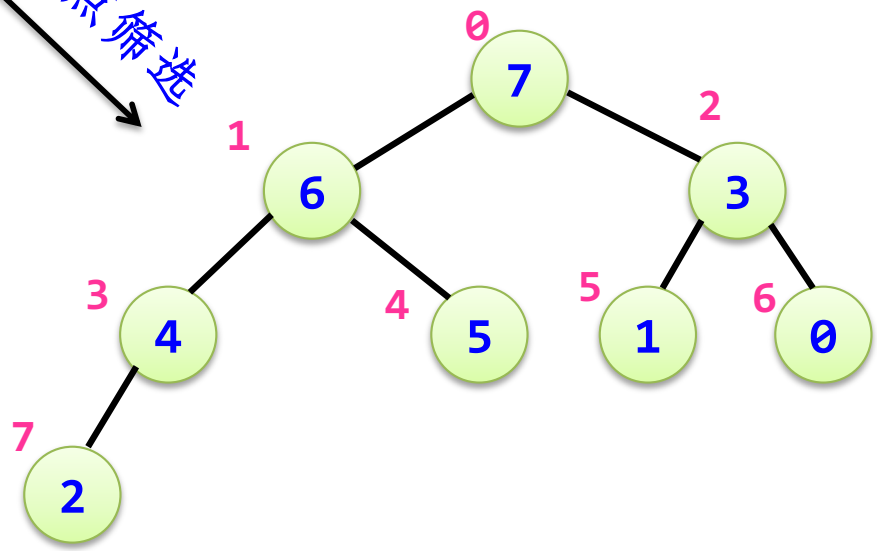
第2趟排序



0 6 7 4 5 1 3 2 8 9

输出8（归位）

从根结点筛选



其他各趟排序依此进行

最终结果：

0 1 2 3 4 5 6 7 8 9

```
void HeapSort(vector<int>& R,int n)
{  for (int i=n/2-1;i>=0;i--)
      siftDown(R,i,n-1);
   for (int i=n-1;i>0;i--)
   {  swap(R[0],R[i]);
      siftDown(R,0,i-1);
   }
}
```

3．算法分析

❶ 对高度为h的堆，一次“筛选”所需进行的关键字比较的次数至多为$2(h-1)$。

❷ 对n个关键字，建成高度为h（$=\lfloor\log_2 n\rfloor+1$）的堆，所需进行的关键字比较的次数不超过$4n$。

❸ 调整“堆顶” $n-1$ 次，总共进行的关键字比较的次数不超过：

$$2\ (\lfloor\log_2(n-1)\rfloor + \lfloor\log_2(n-2)\rfloor + \cdots + \log_2 2) < 2n(\lfloor\log_2 n\rfloor)$$

堆排序的时间复杂度为$O(n\log n)$。

空间复杂度为$O(1)$，不稳定。

$$O(\sum_{j=1}^{h-1} 2^{j-1}(h-j+1))=O(\sum_{k=2}^{h} k2^{h-k})=O(2^h \sum_{k=2}^{h} k2^{-k}) = O(2^h) = O(n)$$

示例

设有1000个无序的整数，希望用最快的速度挑选出其中前10个最大的元素，最好选用（　）排序方法。

A.冒泡排序　　B.简单选择排序

C.堆排序　　D.直接插入排序

$n=1000$，$k=10$

- 冒泡排序的大致时间：kn
- 堆排序的大致时间：$4n+k\log_2 n$。

数据结构经典算法的启示

优先队列就是采用堆实现的！

各种内排序方法的比较和选择

排序方法	时间复杂度			空间复杂度	稳定性
	平均情况	最坏情况	最好情况		
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定

09. 对由相同的 n 个整数构成的二叉排序树和小根堆，下列说法中不正确的是（ ）。

A. 二叉排序树的高度大于或等于小根堆的高度

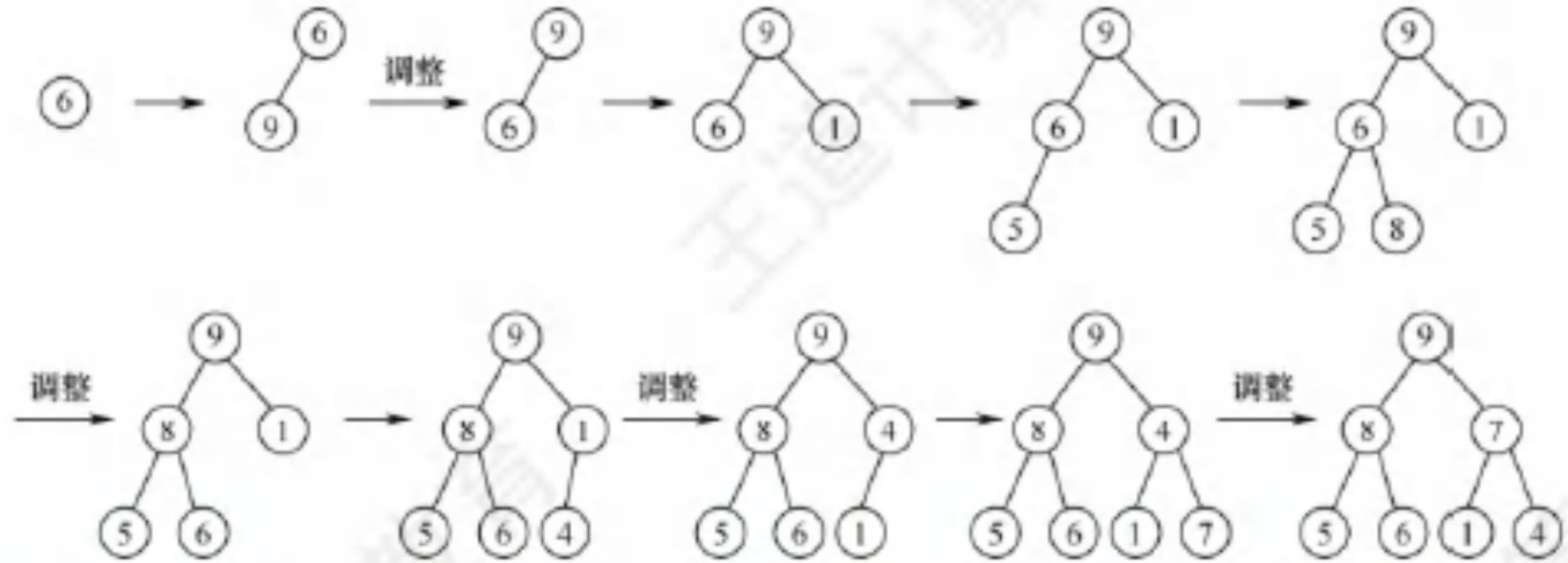
B. 对二叉排序树进行中序遍历可以得到从小到大的序列

C. 从小根堆的根结点到任意叶结点的路径构成从小到大的序列

D. 对小根堆进行层序遍历可以得到从小到大的序列

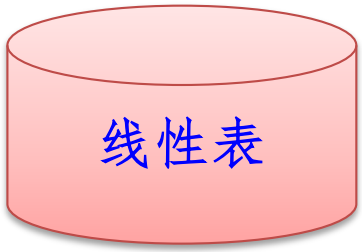
20.【2021 统考真题】将关键字 6,9,1,5,8,4,7 依次插入初始为空的大根堆 H，得到的 H 是(　)。

A. 9,8,7,6,5,4,1　　B. 9,8,7,5,6,1,4　　C. 9,8,7,5,6,4,1　　D. 9,6,7,5,8,4,1



注意，给定序列依次插入空堆的结果与给定序列直接调整成堆的结果是不同的，最终得到的堆的形式也不同。若对序列 6, 9, 1, 5, 8, 4, 7 直接调整成堆，则会误选 C 项。

10.4.3 堆数据结构

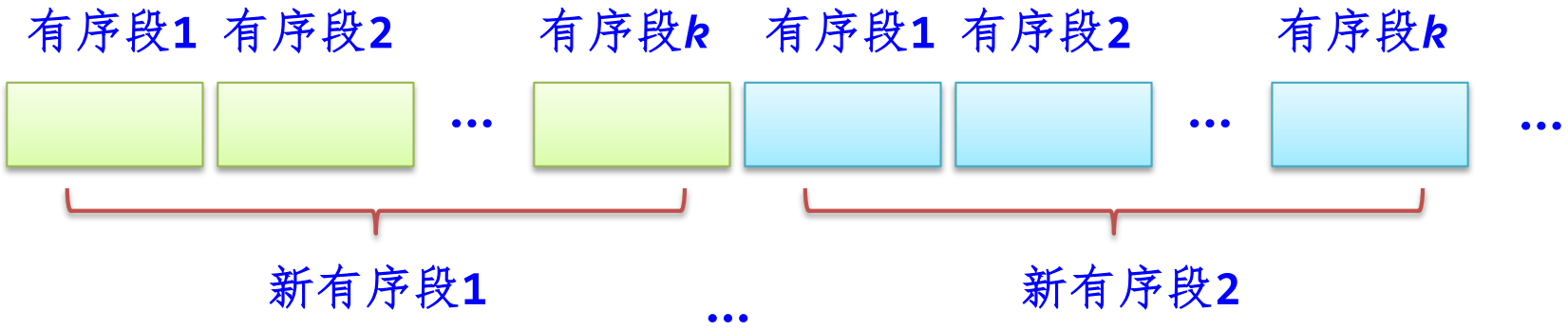


- append(e)：向堆中插入元素*e*。
- pop()：删除堆顶元素并且返回该元素。
- gettop()：取除堆顶元素。
- empty()：判断堆是否为空。

用*R*[0..*n*-1]存放一个堆，即（*R*，*n*）

10.5 归并排序

基本思路（k路归并）



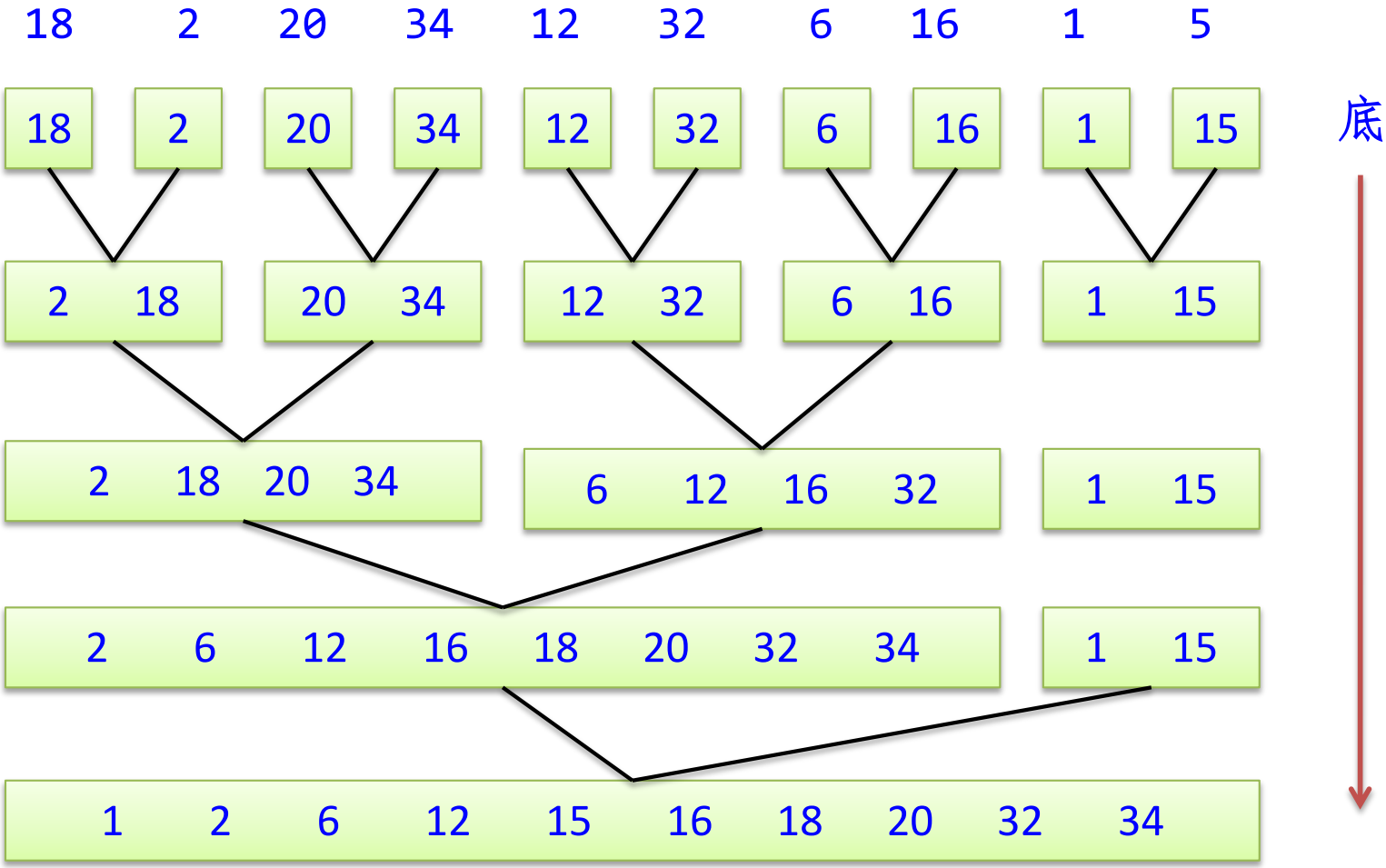
主要的归并排序方法：

二路归并排序

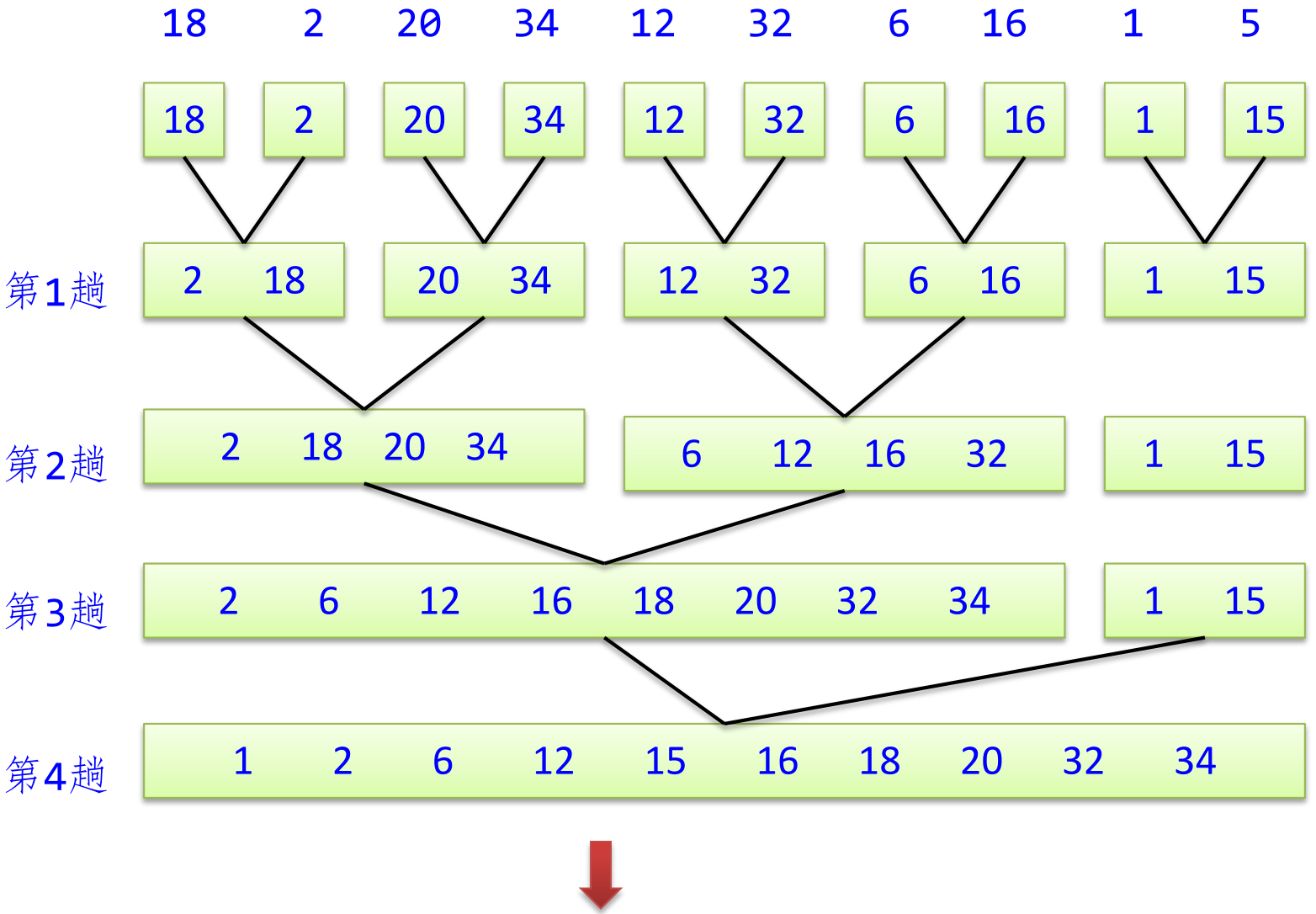
10.5.1 自底向上的二路归并排序

1. 排序思路

【例10.8】



归并树



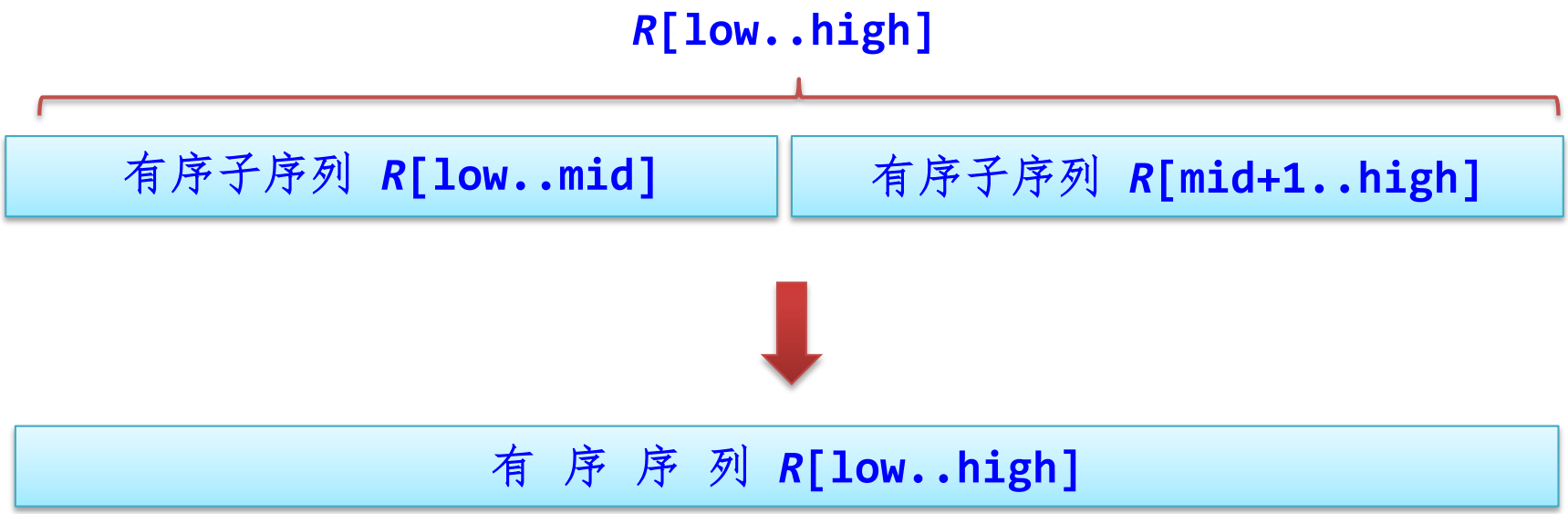
- 有清晰的趟数（同一趟产生的归并段优先归并）
- 归并树高度$h=\lceil \log_2 n \rceil+1$
- 归并的趟数$=h-1$

➡ 平衡归并

2．排序算法设计

1）二路归并算法

基础：将两个位置相邻的有序子序列归并为一个有序序列。

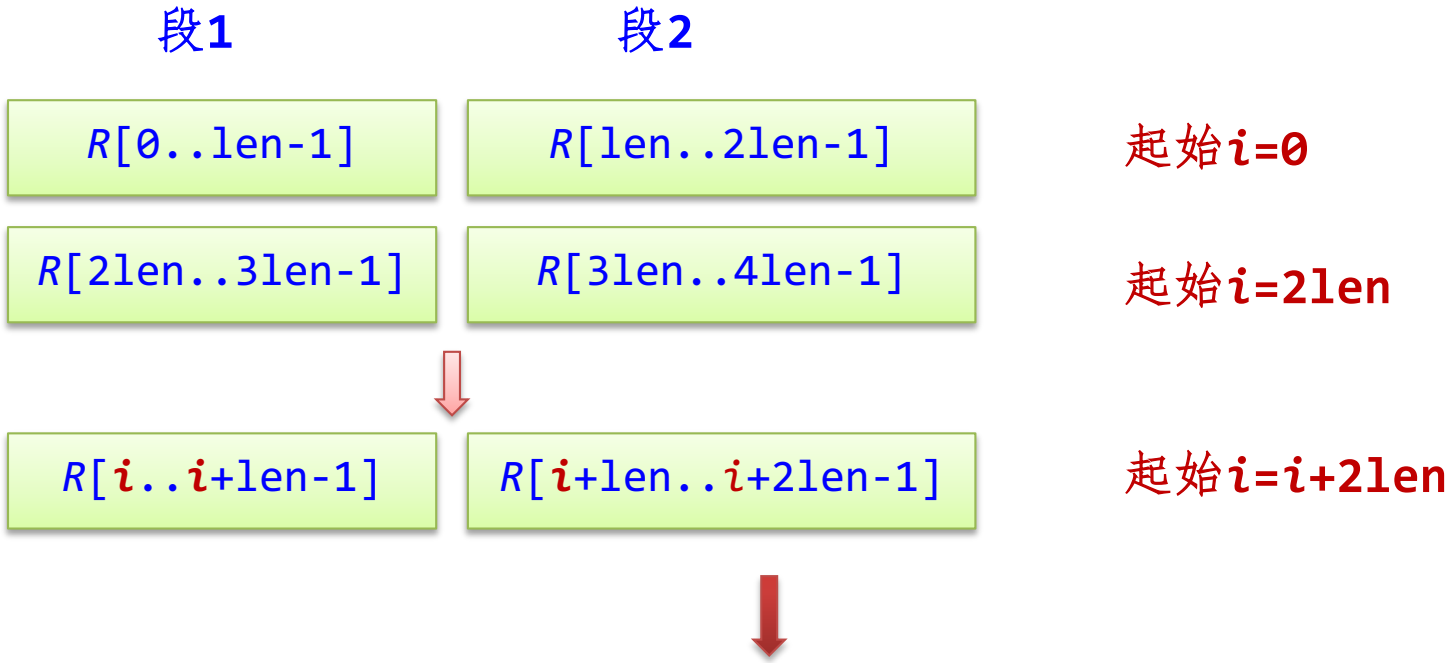


```
void Merge(vector<int>& R,int low,int mid,int high)
//将R[low..mid]和R[mid+1..high]两个有序段归并为一个有序段R[low..high]
{  vector<int> R1;
   R1.resize(high-low+1);          //设置R1的长度为high-low+1
   int i=low,j=mid+1,k=0;          //k是R1的下标,i、j分别为第1、2段的下标
   while (i<=mid && j<=high)       //在第1段和第2段均未扫描完时循环
      if (R[i]<=R[j])              //将第1段中的元素放入R1中
      {  R1[k]=R[i];
         i++; k++;
      }
      else                         //将第2段中的元素放入R1中
      {  R1[k]=R[j];
         j++; k++;
      }
   while (i<=mid)                  //将第1段余下部分复制到R1
   {  R1[k]=R[i];
      i++; k++;
   }
   while (j<=high)                 //将第2段余下部分复制到R1
   {  R1[k]=R[j];
      j++; k++;
   }
   for (k=0,i=low;i<=high;k++,i++) //将R1复制回R中
      R[i]=R1[k];
}
```

空间复杂度为O(high-low+1)

2）一趟二路归并排序

有序子表长度为len ⇨ R[0..n-1]中共分为⌈n/len⌉个有序的子表



- 段2的尾元素序号i+2len-1<n ⇨是满的（两个段均含len个元素）
- i+len-1<n-1（或者i+len<n） ⇨ 剩余两个有序子表
- 否则说明仅剩余一个有序子表（第2段为空），不趟不参与归并

- 段2的尾元素序号$i+2len-1<n$ ⇨是满的（两个段均含len个元素）
- $i+len-1<n-1$（或者$i+len<n$） ⇨ 剩余两个有序子表
- 否则说明仅剩余一个有序子表（第2段为空），不趟不参与归并

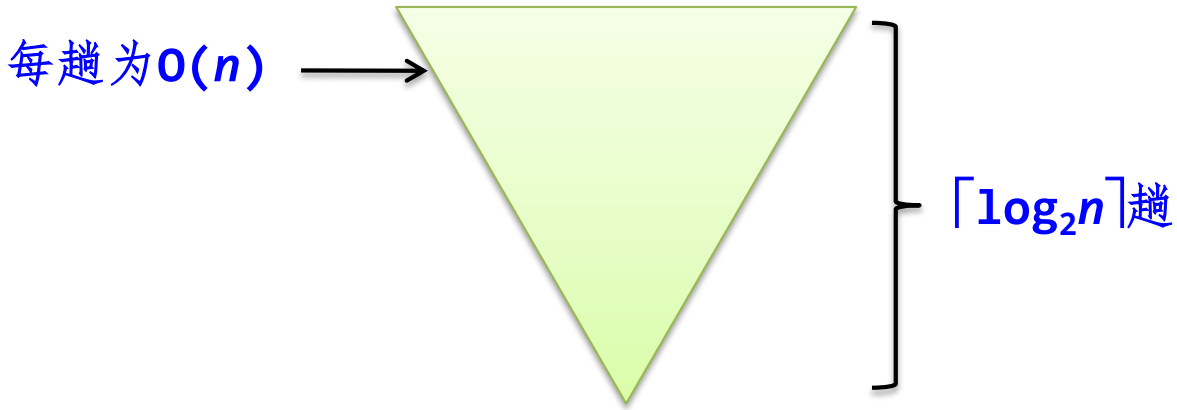
```
void MergePass(vector<int>& R,int length)
//对整个数序进行一趟归并
{  int n=R.size(),i;
   for (i=0;i+2*length-1<n;i+=2*length) //归并length长的两相邻子表
       Merge(R,i,i+length-1,i+2*length-1);
   if (i+length<n)                      //余下两个子表,后者长度小于length
       Merge(R,i,i+length-1,n-1);       //归并这两个子表
}
```

3）二路归并排序

```
void MergeSort1(vector<int>& R,int n)          //自底向上的二路归并排序
{  for (int length=1;length<n;length=2*length) //进行log2n趟归并
     MergePass(R,length);
}
```

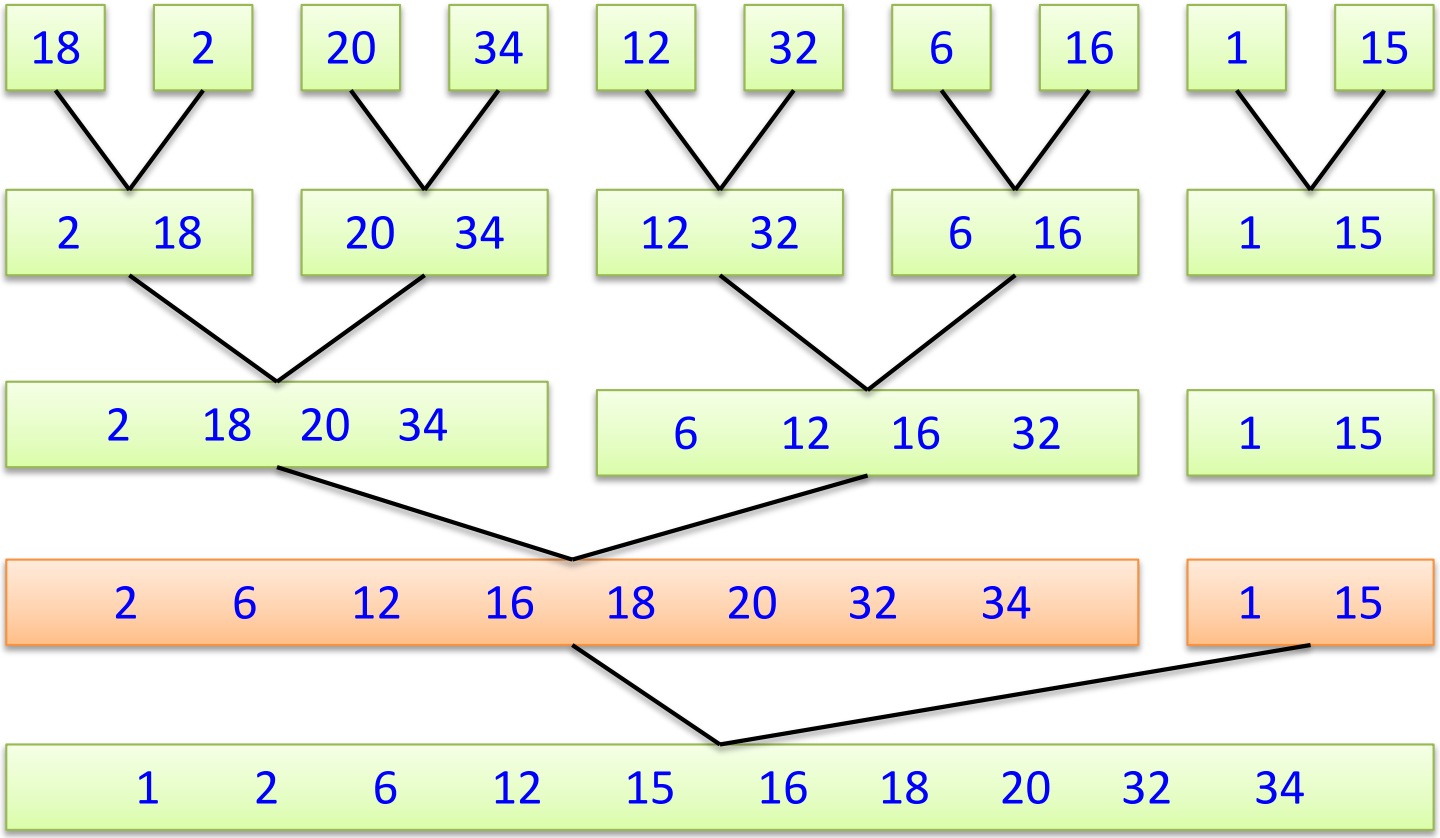
3．算法分析

- 二路归并排序中，长度为n的排序表需做$\lceil \log_2 n \rceil$趟，对应的归并树高度为$\lceil \log_2 n \rceil$，每趟归并时间为$O(n)$。

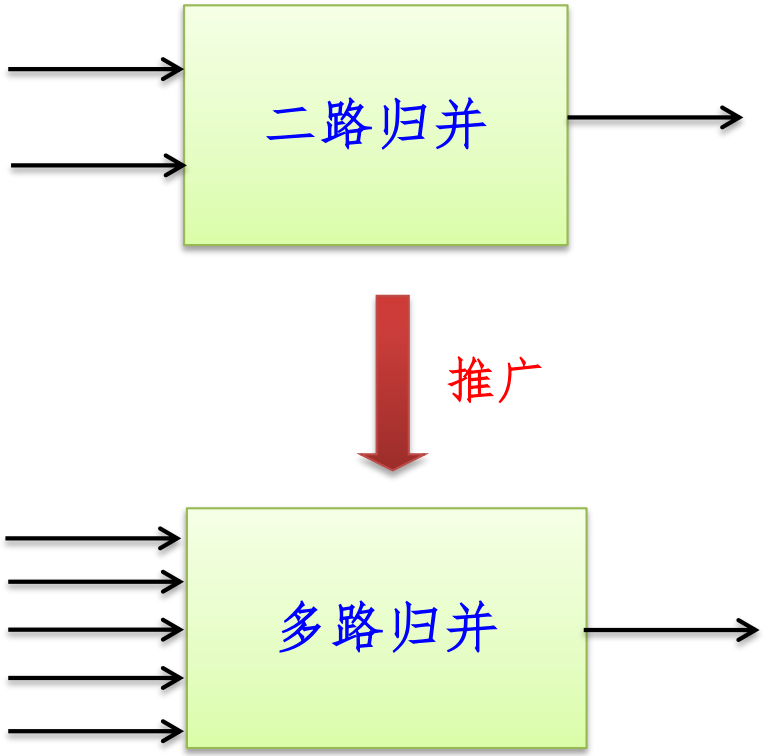


时间复杂度的最好、最坏和平均情况都是$O(n\log_2 n)$。

- 归并排序过程中每次调用Merge都需要使用局部数组*R1*，但执行完后其空间被释放，但最后一趟排序一定是全部*n*个元素参与归并，所以总的辅助空间复杂度为O(*n*)。



扩展



- 三路归并的归并树的高度为$\lceil \log_3 n \rceil$，同样一次三路归并的时间为$O(n)$，所以三路归并排序的时间复杂度为$O(n\log_3 n)$。
- 而$n\log_3 n = n\log_2 n / \log_2 3$，即$O(n\log_3 n) = O(n\log_2 n)$，也就是说，三路归并排序与二路归并排序的时间复杂度相同。

10.5.2 自顶向下的二路归并排序

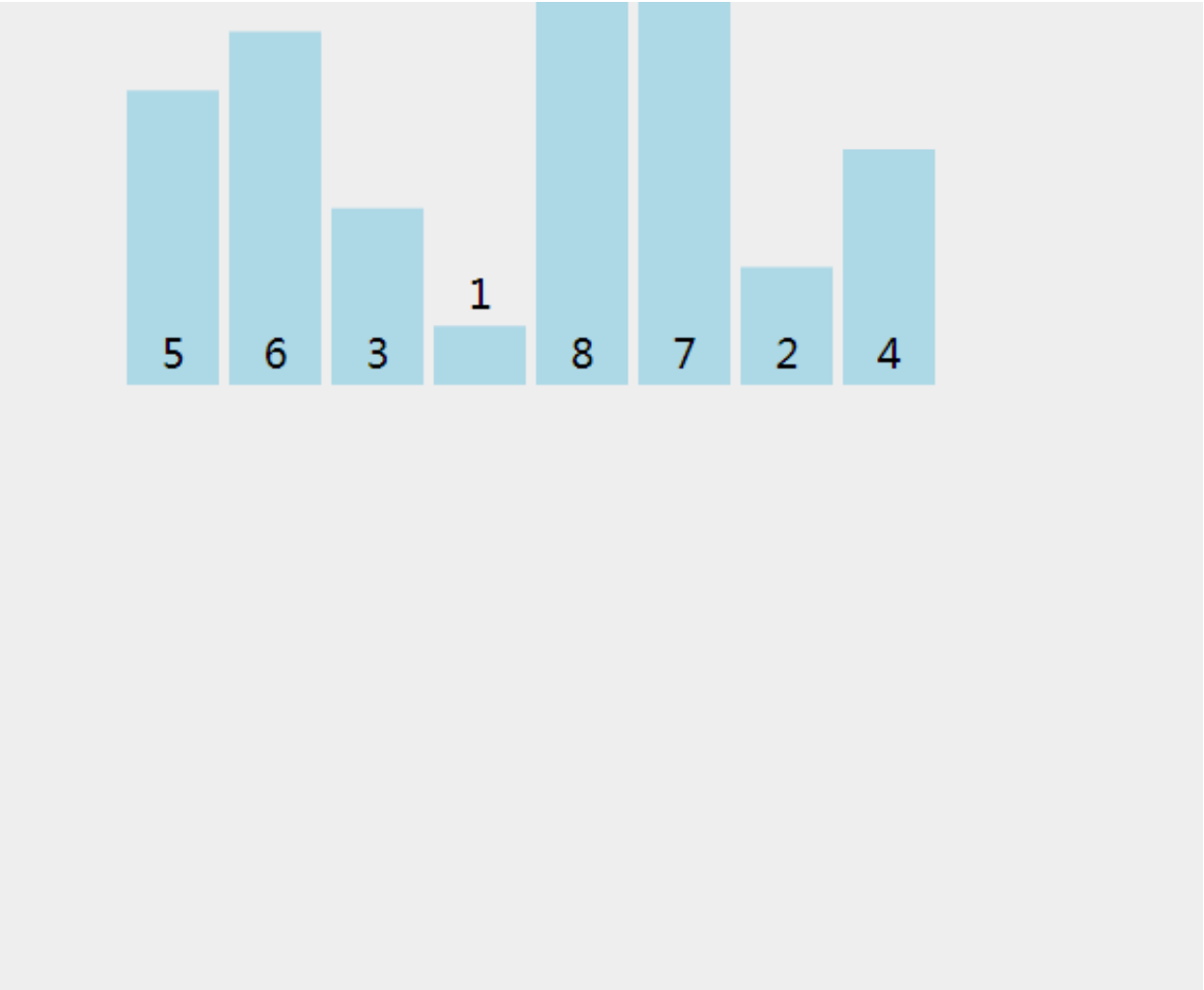
- 排序区间是R[s..t]（为大问题），当其长度为0或者1时，本身就是有序的，不做任何处理。
- 否则，其中间位置m，采用相同方法对R[s..m]和R[m+1..t]排好序（分解为两个小问题），再调用前面的二路归并算法Merge(s, m, t)得到整个有序表（合并）。

```
f(R, s, t) ≡ 不做任何事情          当R[s..t]为空或者仅有一个元素时
f(R, s, t) ≡ m=(s+t)/2;                  其他情况
             f(R, s, m); f(R, m+1, t);
             Merge(s, m, t);
```

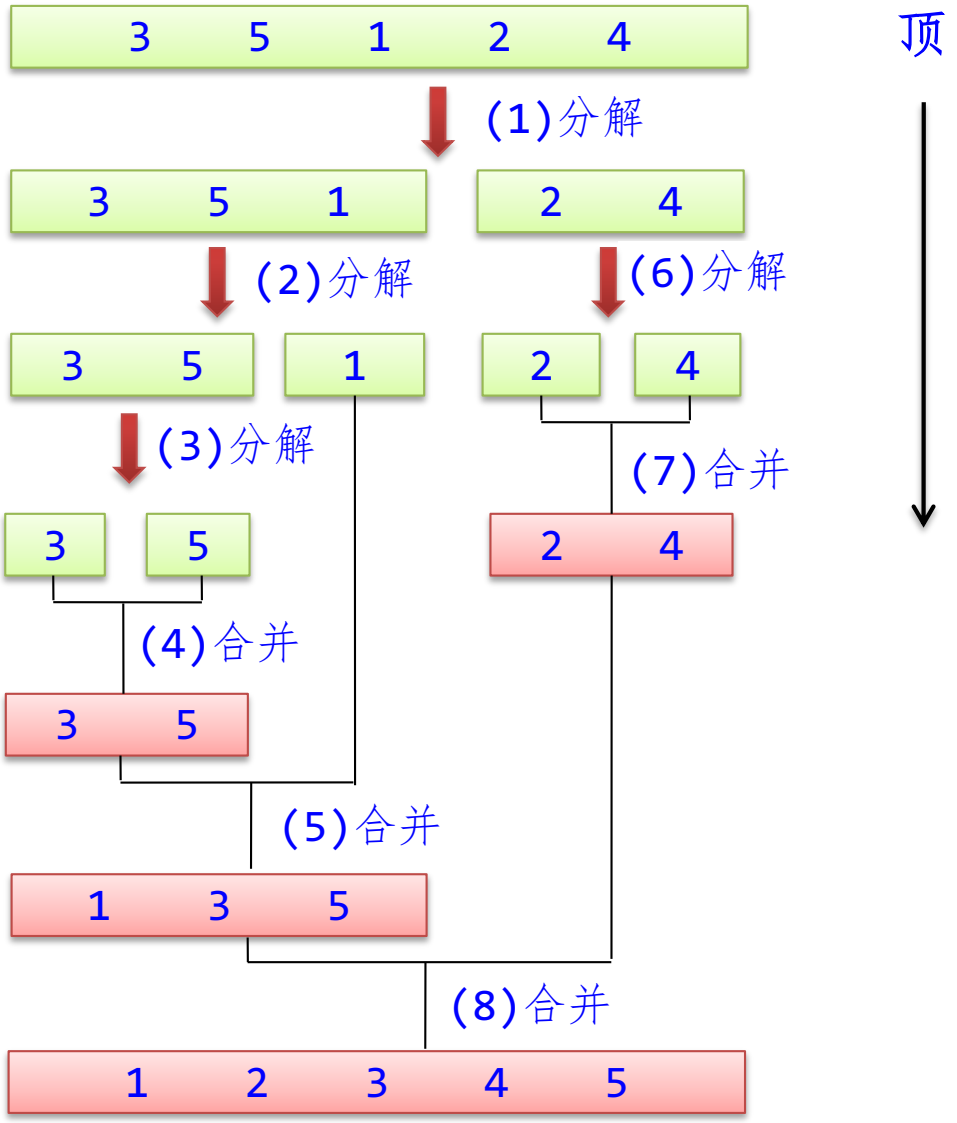
```
void MergeSort2(vector<int>& R,int n)  //自顶向下的二路归并排序
{
   _MergeSort2(R,0,n-1);
}

void _MergeSort2(vector<int>& R,int s,int t) //被MergeSort2调用
{  if (s>=t) return;                        //R[s..t]的长度为0或者1时返回
   int m=(s+t)/2;                          //取中间位置m
   _MergeSort2(R,s,m);                     //对前子表排序
   _MergeSort2(R,m+1,t);                   //对后子表排序
   Merge(R,s,m,t);                         //将两个有序子表合并成一个有序表
}
```

递归二路归并排序方法

递归二路归并排序



【例10.9】设排序序列有5个元素，其关键字分别为（3，5，1，2，4）。说明采用自顶向下二路归并排序方法进行排序的过程。



- 设R[0..n-1]排序的时间为T(n)，当n>1时，_MergeSort2(0,n/2)和_MergeSort2(n/2+1,n-1)两个子问题的时间均为T(n/2)，而Merge的时间为O(n)。
- 对应的递推式如下：

$$
\begin{cases}
T(n)=1 & \text{当 } n=1 \\
T(n)=2T(n/2)+n & \text{当 } n>1
\end{cases}
$$

$$\Downarrow$$

$$T(n)=O(n\log_2 n)$$

- 设R[0..n-1]排序的空间为S(n)，当n>1时，两个子问题的空间均为S(n/2)，而Merge的空间为O(n)。但_MergeSort2(0,n/2)求解完后栈空间释放，被_MergeSort2(n/2+1,n-1)重复使用。
- 对应的递推式如下：

$$S(n)=1 \quad \text{当} n=1$$
$$S(n)=S(n/2)+n \quad \text{当} n>1$$

$$S(n)=O(n)$$

19. 【2022 统考真题】使用二路归并排序对含 n 个元素的数组 M 进行排序时，二路归并排序的功能是（　）。

A. 将两个有序表合并为一个新的有序表

B. 将 M 划分为两部分，两部分的元素个数大致相等

C. 将 M 划分为 n 个部分，每个部分中仅含有一个元素

D. 将 M 划分为两部分，一部分元素的值均小于另一部分元素的值

10.6 基数排序

1．排序思路

- 基数排序是一种借助于多关键字排序的思想对单关键字排序的方法。
- 所谓多关键字是指讨论元素中含有多个关键字，假设多个关键字分别为k^1、k^2、…、k^d，称k^1是第一关键字，k^d是第d个关键字。
- 基数排序就是利用多关键字排序思路，只不过将元素中的单个关键字分为多个位，每个位看成一个关键字。

$(k_j^{d-1}, k_j^{d-2}, \cdots, k_j^1, k_j^0)$

- 一般地，在基数排序中元素R[i]的关键字R[i].key是由d位数字组成，即$k^{d-1}k^{d-2}\cdots k^0$，每一个数字表示关键字的一位，其中k^{d-1}为最高位，k^0是最低位，每一位的值都在$0 \leqslant k^i < r$范围内，其中，r称为基数。
- 例如，对于二进制数r为2，对于十进制数r为10。
- 假设k^{d-1}是最重要位，k^0是最不重要位，应该从最低位开始排序，称为最低位优先（LSD）。
- 反之，若k^{d-1}是最不重要位，k^0是最重要位，应该从最高位开始排序，称为最高位优先（MSD）。

最低位优先排序

假设线性表由元素序列a_0、a_1、…、a_{n-1}构成，每个结点a_j的关键字由d元组

$$(k_j^{d-1},\ k_j^{d-2},\ \cdots,\ k_j^1,\ k_j^0)$$

其中每个元素值在0到$r-1$之间。排序中使用r个队列Q_0，Q_1，…，Q_{r-1}。

对$i=0$、1、…，$d-1$（从低位到高位），依次做一次“分配”和“收集”：

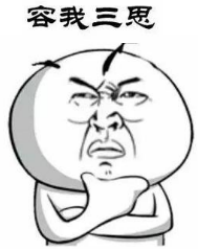
分配

开始时，把Q_0、Q_1、…、Q_{r-1}各个队列置成空队列，然后依次考察线性表中的每一个元素a_j（$j=1$、2、…、n），如果a_j的关键字位 $k_j^i=k$，就把a_j插入到Q_k队列中。

收集

将Q_0、Q_1、…、Q_{r-1}各个队列中的元素依次首尾相接，得到新的元素序列，从而组成新的线性表。

2．排序算法



线性表（a_0、a_1、…、a_{n-1}）采用什么存储结构？

由于在分配和收集中涉及大量元素移动，采用顺序表时效率较低，所以采用单链表存放排序序列，这里采用以*h*为首结点的单链表（不带头结点）

```
template <typename T>
struct LinkNode                          //单链表结点类型
{   T data;                              //存放数据元素
    LinkNode<T>* next;                   //指向下一个结点的域

    LinkNode():next(NULL) {}             //构造函数
    LinkNode(T d):data(d),next(NULL) {} //重载构造函数
};
```

利用第2章单链表类LinkList<T>类模板存放单链表

假设元素关键字均为十进制（***r*=10**）正整数，最大位数为***d***，按递增排序的最低位优先基数排序算法如下：

```
int geti(int key,int r,int i)  //求基数为r的正整数key的第i位
{  int k=0;
   for (int j=0;j<=i;j++)
   {  k=key%r;
      key=key/r;
   }
   return k;
}
```

```
void RadixSort1(LinkList<int>& L,int d,int r)
//最低位优先基数排序算法
{  LinkNode<int>* front[MAXR];          //建立链队队头数组
   LinkNode<int>* rear[MAXR];           //建立链队队尾数组
   LinkNode<int>* p,*t;
   for (int i=0;i<d;i++)                //从低位到高位循环
   {  for (int j=0;j<r;j++)             //初始化各链队首、尾指针
         front[j]=rear[j]=NULL;

      p=L.head->next;
      while (p!=NULL)                   //分配：对于原链表中每个结点循环
      {  int k=geti(p->data,r,i); //提取关键字第k个位并放入第k个链队
         if (front[k]==NULL)          //第k个链队空时，队头队尾均指向p结点
         {  front[k]=p;
            rear[k]=p;
         }
         else                           //第k个链队非空时，p结点进队
         {  rear[k]->next=p;
            rear[k]=p;
         }
         p=p->next;                     //取下一个待排序的结点
    }
```

遍历L的每个结点并分配的相关队列中 ⇨ O(n)

```
    LinkNode<int>* h=NULL;      //重新用h来收集所有结点
    for (int j=0;j<r;j++)       //收集：对于每一个链队循环
       if (front[j]!=NULL)      //若第j个链队是第一个非空链队
       {  if (h==NULL)
          {  h=front[j];
             t=rear[j];
          }
          else                  //若第j个链队是其他非空链队
          {  t->next=front[j];
             t=rear[j];
          }
       }
       t->next=NULL;            //尾结点的next域置NULL
       L.head->next=h;
    }
}
```

遍历r个队列并收集产生单链表L ⇨ O(r)

示例

例如(369，367，167，239，237，138，230，139) ⇨ 基数排序

h →369 →367 →167 →239 →237 →138 →230 →139

建立10个队列，*f*为队头，*r*为队尾

❶ 进行第1次分配：按个位

f[0] →230 ← *r*[0]

f[7] →367 →167 →237 ← *r*[7]

f[8] →138 ← *r*[8]

f[9] →369 →239 →139 ← *r*[9]

进行第1次收集

h →230 →367 →167 →237 →138 →369 →239 →139

第1趟排序完毕

- 分配时是按一个一个元素进行的
- 收集时是按一个一个队列进行的

h →230 →367 →167 →237 →138 →369 →239 →139

❷ 进行第2次分配：按拾位

f[3] →230 →237 →138 →239 →139 ← *r*[3]

f[6] →367 →167 →369 ← *r*[6]

进行第2次收集

h →230 →237 →138 →239 →139 →367 →167 →369

第2趟排序完毕

h →230 →237 →138 →239 →139 →367 →167 →369

❸ 进行第3次分配：按百位

f[1] →138 →139 →167 ← *r*[1]

f[2] →230 →237 →239 ← *r*[2]

f[3] →367 →369 ← *r*[3]

进行第3次收集

h →138 →139 →167 →230 →237 →239 →367 →369

第3趟排序完毕

3．算法分析

- 分配为O(*n*)
- 收集为O(*r*)（*r*为“基数”）
- *d*为“分配-收集”的趟数

基数排序的时间复杂度为O(*d*(*n*+*r*))

基数排序的空间复杂度为O(*r*)

基数排序中为什么不需要进行关键字的比较？

18. 【2021 统考真题】设数组 S[]={93,946,372,9,146,151,301,485,236,327,43,892}，采用最低位优先（LSD）基数排序将 S 排列成升序序列。第一趟分配、收集后，元素 372 之前、之后紧邻的元素分别是（ ）。

A. 43, 892　　B. 236, 301　　C. 301, 892　　D. 485, 301

16. 【2013 统考真题】对给定的关键字序列 110, 119, 007, 911, 114, 120, 122 进行基数排序，第二趟分配、收集后得到的关键字序列是（ ）。

A. 007, 110, 119, 114, 911, 120, 122　　B. 007, 110, 119, 114, 911, 122, 120

C. 007, 110, 911, 114, 119, 120, 122　　D. 110, 120, 911, 122, 114, 007, 119

17.【2015 统考真题】下列排序算法中，元素的移动次数与关键字的初始状态无关的是（ ）。

A. 直接插入排序　B. 冒泡排序　C. 基数排序　D. 快速排序

10.7 各种内排序方法的比较和选择

排序方法	时间复杂度			空间复杂度	稳定性
	平均情况	最坏情况	最好情况		
直接插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
折半插入排序	$O(n^2)$	$O(n^2)$	$O(n\log_2 n)$	$O(1)$	稳定
希尔排序	$O(n^{1.58})$			$O(1)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(\log_2 n)$	不稳定
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定
二路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	稳定
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	稳定

考虑1：排序数据项顺序：k_1、k_2还是k_2、k_1？

标识	k_1	k_2
1	80	90
2	70	90
3	80	72
4	70	72

标识	k_1	k_2
4	70	72
2	70	90
3	80	72
1	80	90

越重要的数据项越在后面排序 ⇨ 应为k_2、k_1

【补充】设线性表中每个元素有两个数据项k_1和k_2，现对线性表按以下规则进行排序：先看数据项k_1，k_1值小的在前，大的在后；在k_1值相同的情况下，再看k_2，k_2值，小的在前，大的在后。满足这种要求的排序方法是（　）。

A.先按k_1值进行直接插入排序，再按k_2值进行简单选择排序

B.先按k_2值进行直接插入排序，再按k_1值进行简单选择排序

C.先按k_1值进行简单选择排序，再按k_2值进行直接插入排序

D.先按k_2值进行简单选择排序，再按k_1值进行直接插入排序

考虑1：排序数据项顺序：k_1、k_2还是k_2、k_1？

标识	k_1	k_2
1	80	90
2	70	90
3	80	72
4	70	72

标识	k_1	k_2
4	70	72
2	70	90
3	80	72
1	80	90

越重要的数据项越在后面排序 ⇨ 应为k_2、k_1

考虑2：k_2选择直接插入排序还是简单选择排序？ 稳定性

例如：

标识	k_1	k_2
1	80	90
2	70	90
3	80	72
4	70	72

k_2直接插入排序

标识	k_1	k_2
3	80	72
4	70	72
1	80	90
2	70	90

k_1简单选择排序

标识	k_1	k_2
2	70	90
4	70	72
1	80	90
3	80	72

相对次序改变

相对次序改变

×

标识	k_1	k_2
1	80	90
2	70	90
3	80	72
4	70	72

k_2简单选择排序

标识	k_1	k_2
4	70	72
3	80	72
1	80	90
2	70	90

k_1直接插入排序

标识	k_1	k_2
4	70	72
2	70	90
3	80	72
1	80	90

相对次序不改变

相对次序不改变 ✓

答案为D。先按k_2值进行简单选择排序，再按k_1值进行直接插入排序

应用场景：考试录取中的成绩排序

- 在考试录取工作中，考试和录取按省份分别开展，各省考试人数从几万到一百多万不等，这样规模的录取工作需要计算机来处理，其中最核心的步骤就是将考生按考分进行排序，排序的效率会直接决定录取工作的处理时长。
- 在高考这个实际的例子中，由于考生众多，很容易出现总分同分的现象。为了决定在总分同分时的录取顺序，还需要制定更详细的比较规则。以浙江省为例，在录取时会依次按文化总分、语文数学总分、语文或数学单科成绩、外语成绩、选考科目单科成绩由高到低，以及志愿号由小到大决定录取顺序。
- 从排序的角度来讲，文化总分是排序的主关键字（第一关键字），语文数学总分是次关键字（第二关键字），语文单科成绩是第三关键字，依次类推。这些比较项目全部相同的考生为同位次，当然出现这种情况的可能性就非常小了。

THE END

我现在做的事情会使我更接近目标吗？